
Business Intelligence y Modelado de Datos

Diseña dashboards profesionales y modela datos como un experto

Alex Goyzueta Delgado

alexgoyzueta2018@gmail.com

Business Intelligence y Modelado de Datos

Diseña dashboards profesionales y modela datos como un experto

© 2026 Alex Goyzueta Delgado

Todos los derechos reservados.

Primera edición: 2026

Colección **Data & Analytics Mastery** — Libro 3

Contacto: alexgoyzueta2018@gmail.com

Dedicatoria

A los analistas que pasan de hacer reportes a transformar decisiones.

A los líderes que entienden que sin datos solo son opiniones.

A todos los que creen que un buen dashboard puede cambiar una organización.

Prefacio

En los libros anteriores construiste una base sólida: SQL, Excel, Python y la nube con BigQuery. Ahora es momento de dar el siguiente paso: convertir datos en decisiones.

El Business Intelligence no es solo hacer gráficos bonitos. Es diseñar modelos de datos que reflejen el negocio, definir métricas que importan, construir dashboards que se usen y crear una cultura data-driven en tu organización.

Este libro te guía desde los fundamentos del modelado dimensional hasta la publicación de dashboards corporativos en Power BI Service, pasando por DAX, Power Query y la seguridad a nivel de fila.

Seguiremos usando TechStore como hilo conductor, pero ahora con un enfoque de BI profesional: modelaremos los datos como un verdadero data warehouse, diseñaremos un star schema, construiremos dashboards ejecutivos y automatizaremos la actualización de datos.

Prepárate para pensar como un arquitecto de BI.

Introducción

Cómo usar este libro

Cada capítulo combina teoría y práctica. Al final encontrarás ejercicios. Las soluciones están en el Apéndice A.

¿Qué necesitas?

- Power BI Desktop (gratuito): <https://powerbi.microsoft.com/desktop>
- Base de datos TechStore (SQLite — la misma del Libro 1 y 2)
- (Opcional) Cuenta de Power BI Service para el Proyecto 4

¿Qué aprenderás?

- Fundamentos de Business Intelligence y modelado dimensional
- Power BI Desktop: conexión, modelado, visualizaciones y DAX
- Power Query para ETL y transformación de datos
- Power BI Service: publicación, workspaces, RLS y apps
- Un proyecto completo de BI corporativo

TechStore

TechStore sigue siendo nuestra empresa ficticia: una tienda de electrónica con +6000 pedidos, 500 productos, 250 clientes y 25 empleados. Los datos están en SQLite pero migraremos el modelo a un star schema profesional en Power BI.

Proyectos del libro

Proyecto 1: Fundamentos de BI y Modelado de Datos

Proyecto 2: Power BI — De los Datos al Dashboard

Proyecto 3: DAX Avanzado y Power Query

Proyecto 4: BI Corporativo con Power BI Service

Cada proyecto termina con un checkpoint que integra lo aprendido. Al final, tendrás un sistema de BI completo y publicable.

Capítulo 1: Introducción al Business Intelligence

¿Qué es Business Intelligence?

Business Intelligence (BI) es el conjunto de estrategias, procesos, arquitecturas y tecnologías que transforman datos crudos en información significativa y útil para la toma de decisiones de negocio.

No es solo software. Es una disciplina que combina:

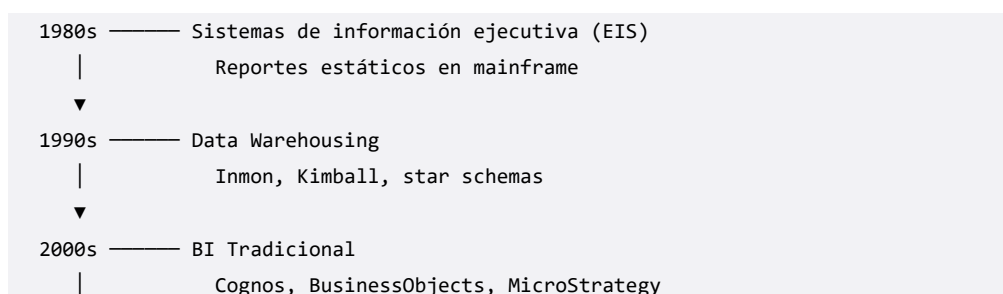
- **Datos:** fuentes internas y externas
- **Tecnología:** herramientas de extracción, transformación, carga y visualización
- **Procesos:** ETL, modelado, actualización, gobernanza
- **Personas:** analistas, arquitectos de BI, consumidores de datos

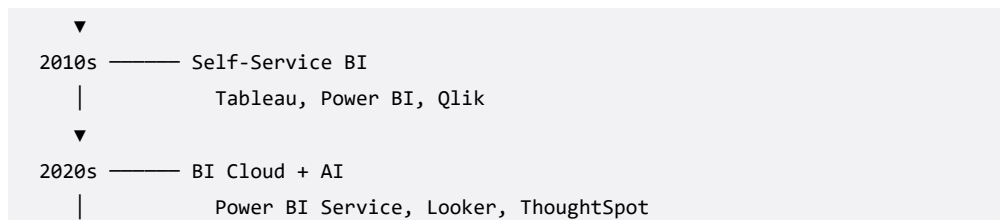
BI vs Analytics

Enfoque	Qué pasó y por qué	Qué pasará y cómo optimizar
Output	Dashboards, reportes, KPIs	Modelos ML, experimentos, recomendaciones
Pregunta	¿Cuánto vendimos este mes?	¿Qué factores predicen la venta?

BI y Analytics no son opuestos: BI responde "qué pasó", Analytics responde "qué pasará". Ambos se complementan.

Evolución histórica del BI





Roles en un equipo de BI

Rol	Responsabilidad
Arquitecto de BI	Diseña la arquitectura de datos, elige herramientas, define estándares
Analista de BI	Construye dashboards, define KPIs, interpreta datos
Data Engineer	Construye y mantiene pipelines ETL/ELT
Consumidor de BI	Usa dashboards para tomar decisiones
Data Steward	Asegura calidad, gobierno y documentación de datos

En este libro te enfocarás en los roles de **Analista de BI** y **Arquitecto de BI**.

El ciclo de vida del BI

1. Identificar necesidades de negocio
 - └ ¿Qué decisiones necesitamos mejorar?
2. Definir KPIs y métricas
 - └ ¿Qué medimos? ¿Cómo lo medimos?
3. Obtener y preparar datos
 - └ ETL/ELT, limpieza, integración
4. Modelar los datos
 - └ Star schema, tablas de hechos y dimensiones
5. Construir dashboards
 - └ Visualizaciones, filtros, interactividad
6. Publicar y compartir
 - └ Power BI Service, permisos, programación
7. Monitorizar y mejorar
 - └ Feedback, nuevas métricas, optimización

El desafío de TechStore

TechStore ha crecido. Ahora tienen:

- Datos en SQLite (transaccional)
- Algunos reportes en Excel
- Datos de ventas en BigQuery (del Libro 2)

Necesitan un **sistema de BI profesional** que unifique todas las fuentes y permita a la gerencia tomar decisiones en tiempo real.

Tu misión en este libro: diseñar, construir y publicar ese sistema.

Ejercicios

1. ¿Cuál es la diferencia entre un reporte operativo y un dashboard estratégico?

2. Identifica 3 decisiones que la gerencia de TechStore podría mejorar con BI
3. ¿Qué perfil de BI eres tú actualmente? ¿A cuál quieres llegar?
4. Investiga: ¿qué es un data-driven organization?
5. ¿Por qué el self-service BI transformó la industria?
6. ¿Qué es un KPI? Nombra 3 ejemplos para TechStore
7. ¿Cuál es la diferencia entre BI tradicional y BI cloud?
8. ¿En qué etapa del ciclo de BI crees que hay más valor agregado?
9. ¿Qué herramientas de BI has usado? ¿Cuál te gustaría aprender?
10. Diseña un diagrama del ciclo de BI para TechStore

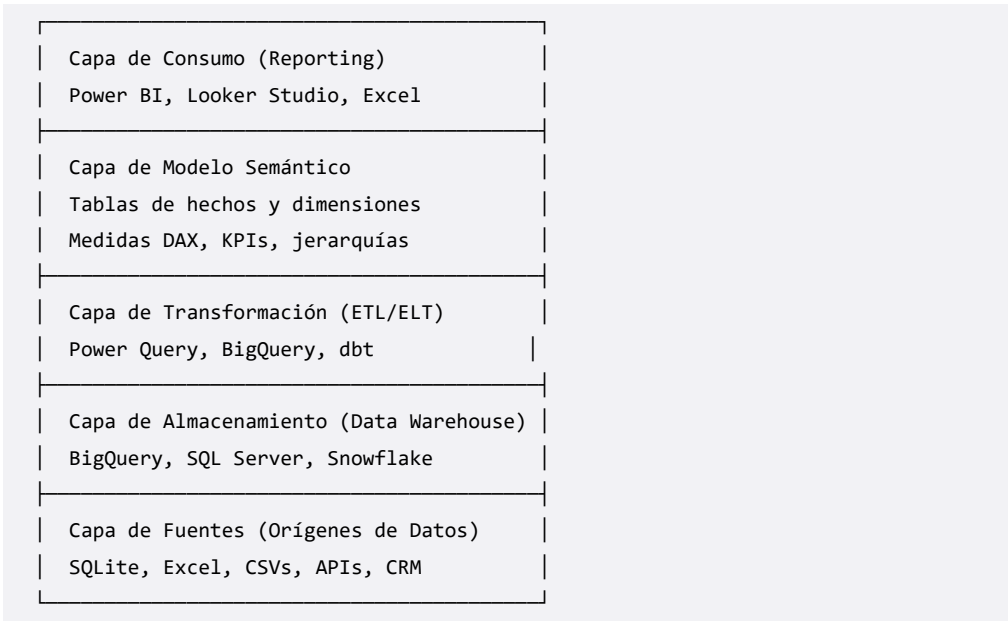
Capítulo 2: Arquitectura de Datos para BI

De fuentes transaccionales a analíticas

Los sistemas transaccionales (OLTP) están optimizados para escribir y leer registros individuales rápidamente. Los sistemas analíticos (OLAP) están optimizados para leer grandes volúmenes de datos agregados.

Característica	OLTP (Transaccional)	OLAP (Analítico)
Uso	Operaciones diarias	Análisis y reporting
Usuarios	Clientes, cajeros, operadores	Ejecutivos, analistas
Operaciones	INSERT, UPDATE, DELETE	SELECT, agregaciones
Datos	Actuales, detallados	Históricos, agregados
Modelo	Normalizado (3NF)	Desnormalizado (Star Schema)
Ejemplo	SQLite de TechStore	Power BI, BigQuery

Arquitectura de BI en capas



Data Warehouse vs Data Mart

	Data Warehouse	Data Mart
Alcance	Toda la organización	Un área o departamento
Datos	Integrados, corporativos	Subconjunto temático
Tamaño	Grande (TB-PB)	Pequeño (GB-TB)
Tiempo	Largo plazo (meses-años)	Corto plazo (semanas-meses)
Ejemplo	DWH corporativo de TechStore	Data Mart de Ventas

Estrategias de construcción:

- **Top-Down (Inmon):** Data Warehouse corporativo primero, luego Data Marts
- **Bottom-Up (Kimball):** Data Marts departamentales primero, luego se integran

Para TechStore, usaremos el enfoque **Bottom-Up (Kimball)**: empezamos con un Data Mart de Ventas.

Staging Area

Antes de cargar datos al Data Warehouse, necesitamos un área de staging:

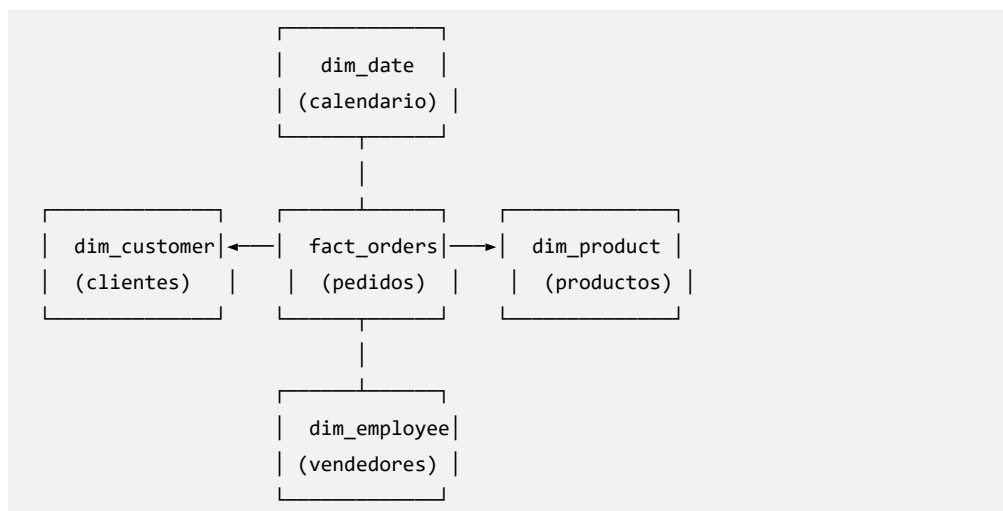
```
-- Ejemplo: tabla staging en BigQuery
CREATE TABLE techstore_staging.orders_staging AS
SELECT * FROM techstore.orders
WHERE DATE(order_date) = CURRENT_DATE();
```

La staging area:

- Almacena datos crudos sin transformar
- Es temporal (se limpia después de procesar)
- Permite validar calidad antes de cargar al modelo final

Esquema de estrella (Star Schema)

El **star schema** es el modelo de datos más usado en BI. Consiste en:



- **Tabla de hechos (fact):** medidas numéricas (total, cantidad), foreign keys a dimensiones
- **Tablas de dimensión (dim):** descriptivas (nombre, ciudad, categoría), texto

Esquema de copo de nieve (Snowflake)

Variante del star schema donde las dimensiones están normalizadas:

```
dim_customer — dim_city — dim_country
dim_product  — dim_category
```

Ventaja: menos redundancia. Desventaja: consultas más lentas (más JOINS).

En Power BI y la mayoría de herramientas, **star schema es la recomendación estándar**.

Tabla de calendario

Una tabla esencial en cualquier modelo de BI es la tabla de calendario (Date Dimension):

```
-- Generar calendario en SQL (BigQuery)
CREATE TABLE techstore.dim_date AS
SELECT
  fecha,
  EXTRACT(YEAR FROM fecha) AS anio,
  EXTRACT(MONTH FROM fecha) AS mes,
  FORMAT_DATE('%B', fecha) AS nombre_mes,
  EXTRACT(QUARTER FROM fecha) AS trimestre,
  CONCAT('Q', EXTRACT(QUARTER FROM fecha), ' ', EXTRACT(YEAR FROM fecha)) AS trimestre_nombre,
  EXTRACT(WEEK FROM fecha) AS semana,
  EXTRACT(DAYOFWEEK FROM fecha) AS dia_semana_num,
  FORMAT_DATE('%A', fecha) AS dia_semana,
  CASE WHEN EXTRACT(DAYOFWEEK FROM fecha) IN (1, 7) THEN TRUE ELSE FALSE END AS es_fin_semana
FROM UNNEST(
  GENERATE_DATE_ARRAY('2023-01-01', '2026-12-31', INTERVAL 1 DAY)
) AS fecha;
```

Granularidad

La **granularidad** define el nivel de detalle de cada fila en la tabla de hechos. Para TechStore:

Tabla de Hechos	Granularidad	Ejemplo
`fact_orders`	Un pedido	1 fila = 1 pedido
`fact_order_items`	Un producto en un pedido	1 fila = 1 línea de pedido
`fact_daily_sales`	Un día	1 fila = 1 día de ventas

Regla: define la granularidad antes de diseñar el modelo.

Ejercicios

1. ¿Cuál es la diferencia entre OLTP y OLAP? Pon ejemplos de cada uno
2. Dibuja el star schema de TechStore con las 5 tablas originales
3. ¿Qué es un Data Mart? ¿Cuál sería el primer Data Mart de TechStore?
4. Explica la diferencia entre Inmon y Kimball en tus palabras
5. ¿Para qué sirve una staging area? ¿Qué datos pondrías ahí?
6. Genera la tabla dim_date en BigQuery (o SQLite) para el rango 2023-2026
7. ¿Cuál es la granularidad de la tabla order_items? ¿Y de orders?

8. ¿Por qué el star schema es recomendado para Power BI?
9. Investiga: ¿qué es un slowly changing dimension (SCD)?
10. Diseña un snowflake schema partiendo del star Schema de TechStore

Capítulo 3: Modelado Dimensional — Hechos y Dimensiones

La teoría de Kimball

Ralph Kimball definió el modelado dimensional como una aproximación para diseñar bases de datos analíticas. Sus principios:

1. **Granularidad**: define el nivel de detalle de cada fila
2. **Dimensiones**: el contexto (quién, qué, cuándo, dónde)
3. **Hechos**: las medidas (cuánto, cuántos)
4. **Hechos atómicos**: almacena al máximo detalle, agrega después

Tablas de hechos (Fact Tables)

Contienen las **métricas** del negocio. Cada fila representa un evento.

```
-- Tabla de hechos: pedidos
CREATE TABLE techstore_bi.fact_orders (
  order_sk INTEGER PRIMARY KEY, -- Surrogate Key
  order_date_sk INTEGER,        -- FK a dim_date
  customer_sk INTEGER,          -- FK a dim_customer
  employee_sk INTEGER,          -- FK a dim_employee
  order_id INTEGER,             -- Natural Key (original)
  total NUMERIC(10,2),          -- Medida
  item_count INTEGER,           -- Medida (derivada)
  status VARCHAR(20)            -- Atributo degenerado
);
```

Tipos de hechos

Tipo	Descripción	Ejemplo
Aditivo	Se puede sumar en todas las dimensiones	total, cantidad
Semi-aditivo	Se suma en algunas dimensiones, no en todas	saldo (no suma en tiempo)
No aditivo	No se puede sumar	precio unitario, porcentaje

Para TechStore:

- `total` aditivo (suma en tiempo, cliente, producto)
- `quantity` aditivo

- `unit\_price` no aditivo (promedio, no suma)

Surrogate Keys

En BI se usan **surrogate keys** (SK o ID artificial) en lugar de las claves naturales:

```
-- En lugar de usar customer_id directamente
customer_sk INTEGER REFERENCES dim_customer(customer_sk)

-- Ventajas:
-- - Independencia del sistema fuente
-- - Manejo de cambios lentos (SCD)
-- - Mejor rendimiento en JOINS
```

Tablas de dimensión (Dimension Tables)

Contienen el **contexto** descriptivo. Son tablas anchas con muchas columnas textuales.

```
CREATE TABLE techstore_bi.dim_customer (
  customer_sk INTEGER PRIMARY KEY,
  customer_id INTEGER,          -- Natural Key
  name VARCHAR(100),
  email VARCHAR(100),
  phone VARCHAR(20),
  city VARCHAR(50),
  country VARCHAR(50),
  registration_date DATE,
  valid_from DATE,             -- Para SCD Tipo 2
  valid_to DATE,
  is_current BOOLEAN
);

CREATE TABLE techstore_bi.dim_date (
  date_sk INTEGER PRIMARY KEY,  -- Formato: 20240115
  full_date DATE,
  anio INTEGER,
  mes INTEGER,
  mes_nombre VARCHAR(20),
  trimestre INTEGER,
  semana INTEGER,
  dia_semana_num INTEGER,
  dia_semana_nombre VARCHAR(20),
  es_fin_semana BOOLEAN
);

CREATE TABLE techstore_bi.dim_product (
  product_sk INTEGER PRIMARY KEY,
  product_id INTEGER,
  name VARCHAR(200),
  category VARCHAR(50),
  supplier VARCHAR(100),
  price NUMERIC(10,2)
);
```

```
CREATE TABLE techstore_bi.dim_employee (
  employee_sk INTEGER PRIMARY KEY,
  employee_id INTEGER,
  name VARCHAR(100),
  position VARCHAR(50),
  hire_date DATE,
  salary NUMERIC(10,2)
);
```

Dimensiones conformadas

Una **dimensión conformada** es una dimensión que se comparte entre múltiples tablas de hechos. Por ejemplo, `dim\_date` se usa en fact_orders y fact_inventory.

En TechStore:

- `dim\_date` se comparte entre todas las fact tables
- `dim\_customer` se comparte entre ventas, devoluciones, marketing

Atributos degenerados

Son atributos de la tabla de hechos que no encajan en una dimensión separada. Ejemplo: `status` o `order\_number`.

Se quedan en la fact table porque:

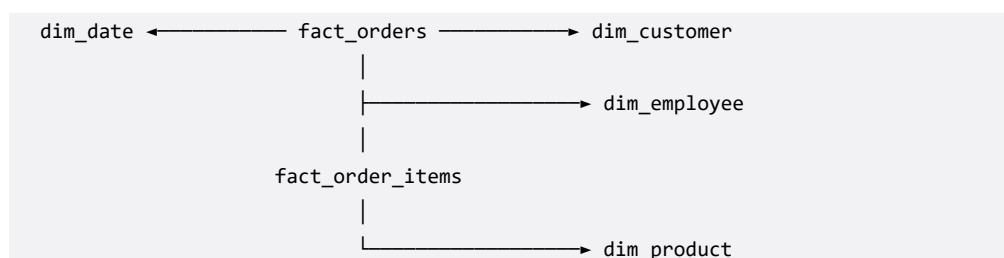
- Tienen alta cardinalidad (muchos valores únicos)
- No comparten contexto entre hechos
- Son referencias operacionales

Hechos sin medidas (Factless Fact Tables)

A veces necesitas una tabla de hechos sin medidas numéricas, solo con foreign keys:

```
-- Evento: cliente visita página web
CREATE TABLE techstore_bi.fact_pageviews (
  date_sk INTEGER,
  customer_sk INTEGER,
  page_sk INTEGER,
  session_id VARCHAR(50),
  PRIMARY KEY (date_sk, customer_sk, page_sk, session_id)
);
```

Diseñando el star schema de TechStore



Estructura final:

```
-- Tabla de hechos: pedidos (granularidad: 1 fila = 1 pedido)
CREATE TABLE techstore_bi.fact_orders (
  order_sk INTEGER PRIMARY KEY,
  date_sk INTEGER NOT NULL,
  customer_sk INTEGER NOT NULL,
  employee_sk INTEGER NOT NULL,
  order_id INTEGER,
  total NUMERIC(10,2),
  item_count INTEGER,
  status VARCHAR(20)
);

-- Tabla de hechos: líneas de pedido (granularidad: 1 fila = 1 producto en 1 pedido)
CREATE TABLE techstore_bi.fact_order_items (
  order_item_sk INTEGER PRIMARY KEY,
  order_sk INTEGER NOT NULL,
  product_sk INTEGER NOT NULL,
  quantity INTEGER,
  unit_price NUMERIC(10,2),
  line_total NUMERIC(10,2)
);
```

Ejercicios

1. Crea la tabla dim_date en SQLite o BigQuery para el rango 2023-2026
2. ¿Cuál es la diferencia entre natural key y surrogate key?
3. Diseña la tabla dim_product con 6 atributos descriptivos
4. ¿Qué es un hecho aditivo? Da 2 ejemplos de TechStore
5. ¿Por qué status es un atributo degenerado?
6. Convierte la tabla orders de SQLite a star schema (fact + dims)
7. ¿Qué es una dimensión conformada? ¿Cuál usarías en TechStore?
8. Investiga SCD Tipo 1, 2 y 3. ¿Cuál usarías para dim_customer?
9. Diseña una factless fact table para seguimiento de campañas de marketing
10. Crea el diagrama completo del star schema de TechStore con todas las tablas

Capítulo 4: KPIs, Métricas y Cuadros de Mando

Métricas vs KPIs

	Métrica	KPI
Definición	Medición cuantitativa	Métrica ligada a un objetivo estratégico
Propósito	Monitorear	Evaluar desempeño vs meta
Frecuencia	Constante	Periódica (diaria, semanal, mensual)
Ejemplo	Número de pedidos	% de crecimiento vs mes anterior
Meta	No siempre tiene target	Siempre tiene un target

Todas las KPIs son métricas, pero no todas las métricas son KPIs.

Diseñando KPIs efectivos

Framework SMART

Criterio	Pregunta	Ejemplo
Specific	¿Qué medimos exactamente?	Ingresos mensuales
Measurable	¿Se puede cuantificar?	\$, %, unidades
Achievable	¿Es alcanzable?	+10% vs realista
Relevant	¿Importa para el negocio?	Directamente ligado a ingresos
Time-bound	¿En qué período?	Este mes vs mes anterior

Jerarquía de KPIs

```
Nivel Estratégico (CEO, Dirección)
├── Ingresos, Rentabilidad, Cuota de mercado
│
├── Nivel Táctico (Gerentes)
│   ├── Ventas por región, Customer Lifetime Value, Rotación
│   │
│   └── Nivel Operativo (Supervisores)
│       ├── Pedidos por hora, Tasa de cancelación, Stock crítico
```

KPIs para TechStore

Área de Ventas

KPI	Fórmula	Frecuencia	Target
Ingresos Mensuales	SUM(total)	Mensual	\$150,000
Ticket Promedio	AVG(total)	Diario	\$320
Tasa de Conversión	Pedidos / Visitantes × 100	Diario	3.5%
Crecimiento vs mes anterior	Mes actual - Mes anterior / Mes anterior	Mensual	+5%
Ventas por empleado	SUM(total) / COUNT(employee)	Mensual	\$25,000

Área de Clientes

KPI	Fórmula	Frecuencia	Target
Clientes Activos	COUNT(DISTINCT customer_id)	Mensual	+10%
Customer Lifetime Value	AVG(total_por_cliente)	Trimestral	\$2,500
Tasa de Retención Clientes que repiten / Total clientes		Trimestral	40%
NPS (Net Promoter Score)	Promotores - Detractores	Trimestral	+50

Área de Operaciones

KPI	Fórmula	Frecuencia	Target
Pedidos por día	COUNT(order_id) / COUNT(DISTINCT date)	Diario	20
Tasa de Cancelación	Pedidos cancelados / Total pedidos	Mensual	< 5%
Días entre pedidos (clientes)	AVG(días entre compras)	Mensual	< 60

Cuadros de Mando (Scorecards)

Un **cuadro de mando** agrupa KPIs por perspectiva. El más famoso es el **Balanced Scorecard** (Kaplan & Norton):

Perspectiva	Pregunta	KPIs TechStore
Financiera	¿Cómo nos ven los accionistas?	Ingresos, Margen, Rentabilidad
Cliente	¿Cómo nos ven los clientes?	NPS, Retención, Satisfacción
Interna	¿En qué debemos sobresalir?	Tasa cancelación, Calidad, Eficiencia
Aprendizaje	¿Cómo mejoramos?	Capacitación, Rotación empleados

KPIs en SQL

```
-- Dashboard de KPIs para TechStore
SELECT
  -- Ingresos
  ROUND(SUM(total), 2) AS ingresos_totales,
  ROUND(AVG(total), 2) AS ticket_promedio,
  COUNT(*) AS total_pedidos,
  COUNT(DISTINCT customer_id) AS clientes_unicos,
  -- Métricas calculadas
  ROUND(SUM(total) / COUNT(*), 2) AS ingreso_por_pedido,
  ROUND(SUM(total) / COUNT(DISTINCT customer_id), 2) AS ingreso_por_cliente,
  ROUND(COUNT(*) / COUNT(DISTINCT customer_id), 1) AS pedidos_por_cliente,
```

```
-- Métricas de cancelación
SUM(CASE WHEN status = 'Cancelled' THEN 1 ELSE 0 END) AS pedidos_cancelados,
ROUND(
    SUM(CASE WHEN status = 'Cancelled' THEN 1 ELSE 0 END) * 100.0 / COUNT(*),
    2
) AS tasa_cancelacion_pct
FROM techstore.orders
WHERE order_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY);
```

KPIs en Power BI (DAX)

Estas medidas las implementaremos en el Proyecto 2:

```
Ingresos Totales = SUM(fact_orders[total])

Ticket Promedio = AVERAGE(fact_orders[total])

Total Pedidos = COUNTROWS(fact_orders)

Clientes Únicos = DISTINCTCOUNT(fact_orders[customer_id])

Crecimiento Mensual % =
VAR mes_actual = SUM(fact_orders[total])
VAR mes_anterior = CALCULATE(SUM(fact_orders[total]), PREVIOUSMONTH(dim_date[full_date]))
RETURN DIVIDE(mes_actual - mes_anterior, mes_anterior, 0)
```

Storytelling con KPIs

Un dashboard efectivo no solo muestra números. Cuenta una historia:

1. **Contexto:** ¿qué es normal? (promedio histórico, target)
2. **Tendencia:** ¿mejorando o empeorando? (vs periodo anterior)
3. **Desglose:** ¿qué impulsa el cambio? (por categoría, región)
4. **Acción:** ¿qué hacer? (recomendación basada en datos)

Ejemplo de storytelling

> "Los ingresos de este mes son \$142,000 — un 8% por debajo del target de \$155,000. La caída se concentra en la categoría Portátiles (-15%) debido a problemas con el proveedor TechSupply. Recomendación: activar el proveedor alternativo e intensificar la promoción de Tablets que está 12% sobre target."

Ejercicios

1. Define 5 KPIs para TechStore usando el framework SMART
2. ¿Cuál es la diferencia entre una métrica y un KPI?
3. Crea una consulta SQL que calcule los KPIs de ventas del último trimestre
4. Diseña un Balanced Scorecard para TechStore con 4 perspectivas
5. ¿Qué es el ticket promedio? ¿Cómo calcularlo en SQL? ¿En DAX?
6. Calcula la tasa de cancelación de TechStore en SQL
7. ¿Cuál es el KPI más importante para un CEO? ¿Y para un vendedor?

8. Escribe un storytelling de 3 párrafos basado en datos ficticios de TechStore
9. ¿Qué diferencia hay entre un dashboard y un scorecard?
10. Define un target SMART para el KPI "Clientes Activos"

Checkpoint 1: Modelo Dimensional de TechStore

Objetivo

Has aprendido los fundamentos de BI, arquitectura de datos, modelado dimensional y KPIs. Ahora es momento de aplicar todo: **diseñar e implementar el modelo dimensional de TechStore** como un star schema listo para Power BI.

Requisitos

- Tener techstore.db en la carpeta `codigos/`
- Python con sqlite3 y pandas

Paso 1: Crear la base de datos dimensional

Crea el script `codigos/crear\_modelo\_bi.py` que genere las tablas del star schema:

```
"""
crear_modelo_bi.py
Construye el modelo dimensional (star schema) de TechStore.
Genera un archivo SQLite listo para Power BI.
"""

import sqlite3
import pandas as pd
from datetime import datetime, timedelta

ORIGEN = "../codigos/techstore.db"
DESTINO = "../codigos/techstore_bi.db"

conn_origen = sqlite3.connect(ORIGEN)
conn_destino = sqlite3.connect(DESTINO)

cursor = conn_destino.cursor()

# --- dim_date ---
print("Creando dim_date...")
cursor.execute("""
    CREATE TABLE dim_date (
```

```

        date_sk INTEGER PRIMARY KEY,
        full_date DATE NOT NULL,
        anio INTEGER NOT NULL,
        mes INTEGER NOT NULL,
        mes_nombre TEXT NOT NULL,
        trimestre INTEGER NOT NULL,
        semana INTEGER NOT NULL,
        dia_semana_num INTEGER NOT NULL,
        dia_semana_nombre TEXT NOT NULL,
        es_fin_semana INTEGER NOT NULL
    )
    """
)

fecha_inicio = datetime(2023, 1, 1)
fecha_fin = datetime(2026, 12, 31)
dias = (fecha_fin - fecha_inicio).days + 1

for i in range(dias):
    fecha = fecha_inicio + timedelta(days=i)
    date_sk = int(fecha.strftime("%Y%m%d"))
    cursor.execute("""
        INSERT INTO dim_date VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    """, (
        date_sk,
        fecha.date().isoformat(),
        fecha.year,
        fecha.month,
        fecha.strftime("%B"),
        (fecha.month - 1) // 3 + 1,
        fecha.isocalendar()[1],
        fecha.weekday() + 1,
        fecha.strftime("%A"),
        1 if fecha.weekday() >= 5 else 0
    ))

print(f" {dias} días insertados")

# --- dim_customer ---
print("Creando dim_customer...")
df_customers = pd.read_sql_query("SELECT * FROM customers", conn_origen)
cursor.execute("""
    CREATE TABLE dim_customer (
        customer_sk INTEGER PRIMARY KEY,
        customer_id INTEGER,
        name TEXT,
        email TEXT,
        phone TEXT,
        city TEXT,
        country TEXT,
        registration_date DATE
    )
    """)
)

```

```

for _, row in df_customers.iterrows():
    cursor.execute("""
        INSERT INTO dim_customer VALUES (?, ?, ?, ?, ?, ?, ?, ?)
    """, (
        row["customer_id"],
        row["customer_id"],
        row["name"],
        row["email"],
        row.get("phone", ""),
        row["city"],
        row.get("country", "USA"),
        row.get("registration_date", "2023-01-01")
    ))

# --- dim_product ---
print("Creando dim_product...")
df_products = pd.read_sql_query("SELECT * FROM products", conn_origen)
cursor.execute("""
    CREATE TABLE dim_product (
        product_sk INTEGER PRIMARY KEY,
        product_id INTEGER,
        name TEXT,
        category TEXT,
        supplier TEXT,
        price REAL
    )
""")
for _, row in df_products.iterrows():
    cursor.execute("""
        INSERT INTO dim_product VALUES (?, ?, ?, ?, ?, ?)
    """, (
        row["product_id"],
        row["product_id"],
        row["name"],
        row["category"],
        row.get("supplier", ""),
        row.get("price", 0)
    ))

# --- dim_employee ---
print("Creando dim_employee...")
df_employees = pd.read_sql_query("SELECT * FROM employees", conn_origen)
cursor.execute("""
    CREATE TABLE dim_employee (
        employee_sk INTEGER PRIMARY KEY,
        employee_id INTEGER,
        name TEXT,
        position TEXT,
        hire_date DATE,
        salary REAL
    )
""")

```

```

for _, row in df_employees.iterrows():
    cursor.execute("""
        INSERT INTO dim_employee VALUES (?, ?, ?, ?, ?, ?)
    """, (
        row["employee_id"],
        row["employee_id"],
        row["name"],
        row["position"],
        row.get("hire_date", "2023-01-01"),
        row.get("salary", 0)
    ))

# --- fact_orders ---
print("Creando fact_orders...")
df_orders = pd.read_sql_query("SELECT * FROM orders", conn_origen)
cursor.execute("""
    CREATE TABLE fact_orders (
        order_sk INTEGER PRIMARY KEY,
        date_sk INTEGER NOT NULL,
        customer_sk INTEGER NOT NULL,
        employee_sk INTEGER NOT NULL,
        order_id INTEGER,
        total REAL,
        status TEXT,
        FOREIGN KEY (date_sk) REFERENCES dim_date(date_sk),
        FOREIGN KEY (customer_sk) REFERENCES dim_customer(customer_sk),
        FOREIGN KEY (employee_sk) REFERENCES dim_employee(employee_sk)
    )
""")
for _, row in df_orders.iterrows():
    fecha = pd.to_datetime(row["order_date"])
    date_sk = int(fecha.strftime("%Y%m%d"))
    cursor.execute("""
        INSERT INTO fact_orders (order_sk, date_sk, customer_sk, employee_sk, order_id, total, status)
        VALUES (?, ?, ?, ?, ?, ?, ?)
    """, (
        row["order_id"],
        date_sk,
        row["customer_id"],
        row.get("employee_id", 1),
        row["order_id"],
        row["total"],
        row["status"]
    ))

# --- fact_order_items ---
print("Creando fact_order_items...")
df_items = pd.read_sql_query("SELECT * FROM order_items", conn_origen)
cursor.execute("""
    CREATE TABLE fact_order_items (
        order_item_sk INTEGER PRIMARY KEY,
        order_sk INTEGER NOT NULL,

```

```

        product_sk INTEGER NOT NULL,
        quantity INTEGER,
        unit_price REAL,
        line_total REAL,
        FOREIGN KEY (order_sk) REFERENCES fact_orders(order_sk),
        FOREIGN KEY (product_sk) REFERENCES dim_product(product_sk)
    )
"""
for _, row in df_items.iterrows():
    cursor.execute("""
        INSERT INTO fact_order_items (order_item_sk, order_sk, product_sk, quantity, unit_price, line_total)
        VALUES (?, ?, ?, ?, ?, ?)
    """, (
        row["order_item_id"],
        row["order_id"],
        row["product_id"],
        row["quantity"],
        row["unit_price"],
        row["quantity"] * row["unit_price"]
    ))

conn_destino.commit()
conn_origen.close()
conn_destino.close()
print("Modelo BI creado: techstore_bi.db")

```

Ejecútalo:

```
python codigos/crear_modelo_bi.py
```

Paso 2: Validar el modelo

Crea `codigos/validar\_modelo\_bi.py`:

```

"""
validar_modelo_bi.py
Valida la integridad del modelo dimensional.
"""

import sqlite3

conn = sqlite3.connect("../codigos/techstore_bi.db")
cursor = conn.cursor()

print("=== Validación del Modelo BI de TechStore ===")
print()

# Tablas y filas
tables = ["dim_date", "dim_customer", "dim_product", "dim_employee",
          "fact_orders", "fact_order_items"]

for table in tables:
    count = cursor.execute(f"SELECT COUNT(*) FROM {table}").fetchone()[0]
    print(f" {table}: {count} filas")

```

```

# Integridad referencial (fact_orders -> dims)
print()
print("=== Integridad Referencial ===")
checks = [
    ("fact_orders.date_sk -> dim_date", """
        SELECT COUNT(*) FROM fact_orders f
        LEFT JOIN dim_date d ON f.date_sk = d.date_sk
        WHERE d.date_sk IS NULL
    """),
    ("fact_orders.customer_sk -> dim_customer", """
        SELECT COUNT(*) FROM fact_orders f
        LEFT JOIN dim_customer c ON f.customer_sk = c.customer_sk
        WHERE c.customer_sk IS NULL
    """),
    ("fact_order_items.product_sk -> dim_product", """
        SELECT COUNT(*) FROM fact_order_items i
        LEFT JOIN dim_product p ON i.product_sk = p.product_sk
        WHERE p.product_sk IS NULL
    """),
]
for name, query in checks:
    orphans = cursor.execute(query).fetchone()[0]
    status = "OK" if orphans == 0 else f"Huerfanos: {orphans}"
    print(f" {name}: {status}")

# KPIs básicos
print()
print("=== KPIs Básicos ===")
kpis = [
    ("Total Ingresos", "SELECT ROUND(SUM(total), 2) FROM fact_orders"),
    ("Total Pedidos", "SELECT COUNT(*) FROM fact_orders"),
    ("Ticket Promedio", "SELECT ROUND(AVG(total), 2) FROM fact_orders"),
    ("Clientes Únicos", "SELECT COUNT(DISTINCT customer_sk) FROM fact_orders"),
]
for name, query in kpis:
    value = cursor.execute(query).fetchone()[0]
    print(f" {name}: {value}")

conn.close()
print()
print("Validación completada.")

```

Paso 3: Diseñar el dashboard de KPIs

Basado en el modelo dimensional, diseña en papel (o con lápiz) un dashboard con:

1. **4 KPIs principales** (tarjetas): Ingresos, Pedidos, Ticket Promedio, Crecimiento %
2. **Evolución mensual** de ingresos (gráfico de líneas)
3. **Ventas por categoría** (gráfico de barras)
4. **Top 5 productos** (tabla o barras)

5. **Ventas por empleado** (gráfico de barras horizontales)
6. **Filtros:** rango de fechas, categoría de producto, vendedor

Entregables del Checkpoint 1

- [] Script `crear\_modelo\_bi.py` funcionando y generando `techstore\_bi.db`
- [] Script `validar\_modelo\_bi.py` con todas las validaciones OK
- [] Modelo dimensional con 4 dimensiones y 2 tablas de hechos
- [] Integridad referencial 100% OK
- [] KPIs básicos calculados y documentados
- [] Diseño del dashboard de KPIs (en papel o digital)

Preguntas de reflexión

1. ¿Cuántas dimensiones creaste? ¿Crees que falta alguna?
2. ¿Qué diferencias encuentras entre el modelo original (SQLite transaccional) y el star schema?
3. ¿Por qué usamos surrogate keys en lugar de las claves naturales?
4. ¿Qué KPI crees que será más útil para la gerencia de TechStore?

¡Felicidades! Has construido tu primer modelo dimensional. En el Proyecto 2 lo conectarás a Power BI para crear dashboards profesionales.

Capítulo 5: Power BI Desktop — Primeros Pasos

¿Qué es Power BI?

Power BI es la plataforma de Business Intelligence de Microsoft. Está compuesta por:

Componente	Qué es	Coste
Power BI Desktop	App de escritorio para crear modelos y dashboards	Gratuito
Power BI Service	Portal cloud para publicar, compartir y colaborar	Gratuito (licencia Pro / Premium de pago)
Power BI Mobile	Apps para iOS y Android	Gratuito
Power BI Report Builder	Creador de reportes paginados	Gratuito
Power BI Embedded	Integración en apps propias	Por uso

En este proyecto nos enfocamos en **Power BI Desktop**.

Instalación

1. Ve a <https://powerbi.microsoft.com/desktop>
2. Descarga e instala Power BI Desktop
3. Abre la aplicación

Pantalla inicial

Al abrir Power BI Desktop verás:

- **Panel de navegación** (izquierda): paneles Reporte, Datos, Modelo
- **Lienzo central**: donde construyes visualizaciones
- **Panel de visualizaciones**: tipos de gráficos, campos, filtros
- **Panel de campos**: lista de tablas y columnas de tu modelo
- **Barra de herramientas**: cinta de opciones con pestañas

Las 3 vistas de Power BI

Vista	Icono	Para qué sirve
Reporte		Crear y diseñar visualizaciones
Datos		Ver y explorar datos en tabla
Modelo		Gestionar relaciones entre tablas

Conectar a TechStore

Vamos a conectar Power BI a nuestro star schema:

1. En la cinta de opciones: "Obtener datos" "SQL Server" o "Más..."
2. Selecciona **ODBC** o **SQLite** (necesitarás el conector SQLite ODBC)
3. Alternativa más simple: importar desde CSV

Opción recomendada: Conectar a SQLite

Power BI no tiene conector nativo a SQLite. Usaremos un archivo CSV temporal en su lugar, o mejor aún: **Datos desde Excel**.

Opción práctica: Datos desde CSV

Exporta cada tabla del star schema a CSV:

```
-- En SQLite, exportar cada tabla a CSV
.headers on
.mode csv
.once dim_date.csv
SELECT * FROM dim_date;
```

O mejor, desde Python:

```
import sqlite3
import pandas as pd

conn = sqlite3.connect("techstore_bi.db")
tables = ["dim_date", "dim_customer", "dim_product", "dim_employee",
          "fact_orders", "fact_order_items"]

for table in tables:
    df = pd.read_sql_query(f"SELECT * FROM {table}", conn)
    df.to_csv(f"{table}.csv", index=False)
    print(f"{table}: {len(df)} filas exportadas")
conn.close()
```

En Power BI:

1. "Obtener datos" "Texto/CSV"
2. Selecciona cada archivo CSV
3. Haz clic en "Cargar"

Opción avanzada: Power Query + ODBC

Si tienes el driver ODBC para SQLite, puedes conectarte directamente:

1. "Obtener datos" "ODBC"
2. Selecciona "SQLite3 Datasource"
3. Navega a `techstore\_bi.db`

Crear relaciones en el modelo

1. Ve a la **Vista Modelo** (icono de diagrama a la izquierda)
2. Arrastra para crear relaciones entre:

- `fact\_orders[date\_sk]` `dim\_date[date\_sk]`
 - `fact\_orders[customer\_sk]` `dim\_customer[customer\_sk]`
 - `fact\_orders[employee\_sk]` `dim\_employee[employee\_sk]`
 - `fact\_order\_items[order\_sk]` `fact\_orders[order\_sk]`
 - `fact\_order\_items[product\_sk]` `dim\_product[product\_sk]`
3. Configura cada relación como:
- **Dirección:** Unidireccional (de dimensión a hecho)
 - **Cardinalidad:** Muchos a uno (*:1)
 - **Cruzada:** Aplica filtro en ambas direcciones NO (excepto si necesitas)

Primeros gráficos

Tarjeta de Ingresos Totales

1. Selecciona el visual "Tarjeta" (card)
2. Arrastra `fact\_orders[total]` al campo "Campos"
3. Power BI automáticamente hace SUM

Gráfico de líneas: Ingresos por mes

1. Selecciona "Gráfico de líneas"
2. Eje X: `dim\_date[full\_date]` (o `dim\_date[anio\_mes]`)
3. Eje Y: `fact\_orders[total]` (SUM)

Gráfico de barras: Ventas por categoría

1. Selecciona "Gráfico de barras"
2. Eje Y: `dim\_product[category]`
3. Eje X: `fact\_orders[total]` (SUM)

Guardar el archivo

1. Archivo Guardar como `TechStore\_BI.pbix`
2. El archivo .pbix contiene el modelo, los datos y las visualizaciones

Ejercicios

1. Instala Power BI Desktop si no lo tienes
2. Conecta las 6 tablas del star schema de TechStore a Power BI
3. Crea las relaciones en el modelo y verifica la cardinalidad
4. Crea una tarjeta con el total de pedidos (COUNT de order_sk)
5. Crea un gráfico de líneas con ingresos por mes
6. ¿Qué diferencia ves entre la Vista Reporte, Datos y Modelo?
7. ¿Para qué sirve la dirección de filtro en las relaciones?
8. Agrega una segmentación de datos (slicer) para filtrar por año
9. Crea un gráfico de barras con ventas por categoría
10. Guarda tu archivo como .pbix

Capítulo 6: Modelado de Datos en Power BI

El modelo es el corazón del dashboard

En Power BI, el modelo de datos determina:

- Qué cálculos son posibles
- Qué tan rápido responden las visualizaciones
- Qué filtros afectan a qué medidas

Un modelo mal diseñado produce medidas incorrectas y dashboards lentos.

Tipos de modelos en Power BI

Import (Importación)

Los datos se copian a la memoria de Power BI. Rápido, pero limitado a RAM disponible.

Ventajas: velocidad, funcionalidad completa de Power Query y DAX
Desventajas: límite de RAM (1 GB en Power BI Desktop / Pro)
Ideal para: datos < 1 GB, dashboards interactivos

DirectQuery

Power BI traduce las consultas al lenguaje de la fuente (SQL).

Ventajas: datos siempre actualizados, sin límite de tamaño
Desventajas: más lento, limitaciones en DAX y Power Query
Ideal para: BigQuery, SQL Server, datos grandes en tiempo real

Composite Model

Mezcla tablas Import y DirectQuery en el mismo modelo.

Ventajas: lo mejor de ambos mundos
Desventajas: complejidad, ciertas limitaciones
Ideal para: modelos avanzados

Para TechStore, usaremos **Import** (los datos caben en RAM).

Cardinalidad de relaciones

Tipo	Símbolo	Ejemplo	Descripción
Muchos a 1	*:1	fact_orders → dim_date	Muchos pedidos tienen una fecha
1 a Muchos	1:*	dim_date → fact_orders	Una fecha tiene muchos pedidos
1 a 1	1:1	dim_customer → fact_orders (raro)	Un cliente por fila
Muchos a Muchos	*:*	Con tabla puente	Casos avanzados

Regla de oro del modelado: todas las tablas de hechos se relacionan con las dimensiones en *:1. Las dimensiones no se relacionan entre sí (tablas huérfanas en el modelo).

Dirección de filtro (Cross Filter Direction)

Dirección	Efecto	Cuándo usarla
Simple (← o →)	La dimensión filtra al hecho	Siempre por defecto
Ambas (↔)	También el hecho filtra a la dimensión	Solo cuando es necesario (cuidado)

Ejemplo de filtro bidireccional

Si `fact\_orders` `dim\_product` y queremos filtrar `dim\_product` por `dim\_employee`:

```
dim_employee → fact_orders → dim_product
(bidireccional)
```

Tablas puente (Bridge Tables)

A veces necesitas una relación muchos a muchos. Ejemplo: un producto puede tener múltiples categorías o un cliente múltiples direcciones.

```
-- Tabla puente: producto-categoría
CREATE TABLE bridge_product_category (
  product_id INTEGER,
  category_id INTEGER,
  PRIMARY KEY (product_id, category_id)
);
```

En Power BI:

- `dim\_product` : `bridge\_product\_category` : `dim\_category`
- Ambas relaciones deben ser unidireccionales

Jerarquías

Power BI permite crear jerarquías dentro de una dimensión:

```
dim_date
├── Año
│   ├── Trimestre
│   │   ├── Mes
│   │   │   ├── Día
│   │   │   └── ...
│   │   └── ...
│   └── ...
└── ...
```

```

dim_product
├── Categoría
│   └── Producto
└── ...

```

Para crear una jerarquía en Power BI:

1. En el panel Campos, haz clic derecho en `dim\_date[anio]`
2. "Nueva jerarquía"
3. Agrega `trimestre`, `mes`, `full\_date` en ese orden

Columnas calculadas vs Medidas

	Columna Calculada	Medida
Cuándo	Al cargar datos	Al consultar
Almacenamiento	En RAM	No almacena (calcula en consulta)
Contexto	Fila por fila	Dinámico (filtros, segmentaciones)
Uso típico	Columnas derivadas (edad, categoría)	Agregaciones (SUM, AVG, COUNT)
Rendimiento	Consume RAM	Más eficiente

Cuándo usar cada una

Usa **columna calculada** cuando:

- Necesitas un valor por fila (ej: nombre completo)
- La columna se usará como filtro o segmentación
- Necesitas ordenar por una columna personalizada

Usa **medida** cuando:

- Necesitas una agregación (SUM, COUNT, AVERAGE)
- El valor depende del contexto de filtro
- Quieres cálculos dinámicos (YTD, vs mes anterior)

Ejemplo: Crear columna calculada

```

-- En dim_product: margen estimado (columna)
Margen Estimado = dim_product[price] * 0.4

-- En dim_customer: nombre completo (columna)
Nombre Completo = dim_customer[name]

-- En fact_orders: rango de ticket (columna)
Rango Ticket = SWITCH(
    TRUE(),
    fact_orders[total] >= 1000, "Premium",
    fact_orders[total] >= 500, "Alto",
    fact_orders[total] >= 200, "Medio",
    "Bajo"
)

```

Ejemplo: Crear medidas básicas

```
-- Total ingresos (medida)
Ingresos Totales = SUM(fact_orders[total])

-- Total pedidos (medida)
Total Pedidos = COUNTROWS(fact_orders)

-- Ticket promedio (medida)
Ticket Promedio = AVERAGE(fact_orders[total])

-- Clientes únicos (medida)
Clientes Activos = DISTINCTCOUNT(fact_orders[customer_sk])
```

Buenas prácticas de modelado

1. **Star schema siempre:** evita snowflake en Power BI
2. **Nombres claros:** usa nombres descriptivos en español
3. **Ocultar columnas clave:** en el panel Campos, oculta surrogate keys (date_sk, customer_sk)
4. **Organiza carpetas:** agrupa medidas en carpetas
5. **Fecha única:** una sola tabla de calendario para todo el modelo
6. **Evita bidireccional:** salvo que sea estrictamente necesario

Ejercicios

1. ¿Cuál es la diferencia entre Import y DirectQuery? ¿Cuál usas para TechStore?
2. Crea una jerarquía de fecha en dim_date (Año Trimestre Mes Día)
3. Crea una columna calculada "Categoría de Precio" en dim_product
4. ¿Cuándo usarías una columna calculada vs una medida?
5. Define 3 medidas básicas: Total Pedidos, Ingreso Promedio, Clientes Únicos
6. Oculta las columnas date_sk, customer_sk, employee_sk en el panel Campos
7. Organiza tus medidas en una carpeta llamada "KPIs Principales"
8. ¿Qué es la cardinalidad : y cómo se maneja en Power BI?
9. Crea una columna en dim_date que indique "Fin de Semana" (Sí/No)
10. ¿Por qué el star schema es la recomendación estándar en Power BI?

Capítulo 7: Visualizaciones y Diseño de Dashboards

Principios de diseño de dashboards

Un buen dashboard no es solo gráficos bonitos. Es comunicación efectiva.

Los 5 principios de Tufte

Principio	Aplicación en Power BI
Integridad	Los datos deben hablar, no decorar
Contexto	Compara con target, histórico, benchmark
Proporción	Escalas correctas (no distorsionar)
Simplicidad	Menos ink = más información
Enfoque	Un dashboard, una historia

Jerarquía visual

Arriba – Izquierda: KPIs principales (lo más importante)
Arriba – Derecha: KPIs secundarios
Centro: Tendencia principal
Abajo: Detalles y desgloses

Tipos de visualización y cuándo usarlas

KPIs y métricas únicas

Visual	Uso	Ejemplo TechStore
Tarjeta	Un solo número	Ingresos totales: \$2.5M
Tarjeta múltiple	Varios KPIs	Ingresos, Pedidos, Clientes
Medidor (Gauge)	vs objetivo	Ingresos vs target \$150K

Comparaciones

Visual	Uso	Ejemplo TechStore
Barra	Comparar categorías	Ventas por categoría
Barra apilada	Composición + comparación	Ventas por categoría y mes
Columna	Evolución temporal	Pedidos por mes
Torta/Donut	Proporciones (máx 5 categorías)	% ventas por categoría

Tendencias

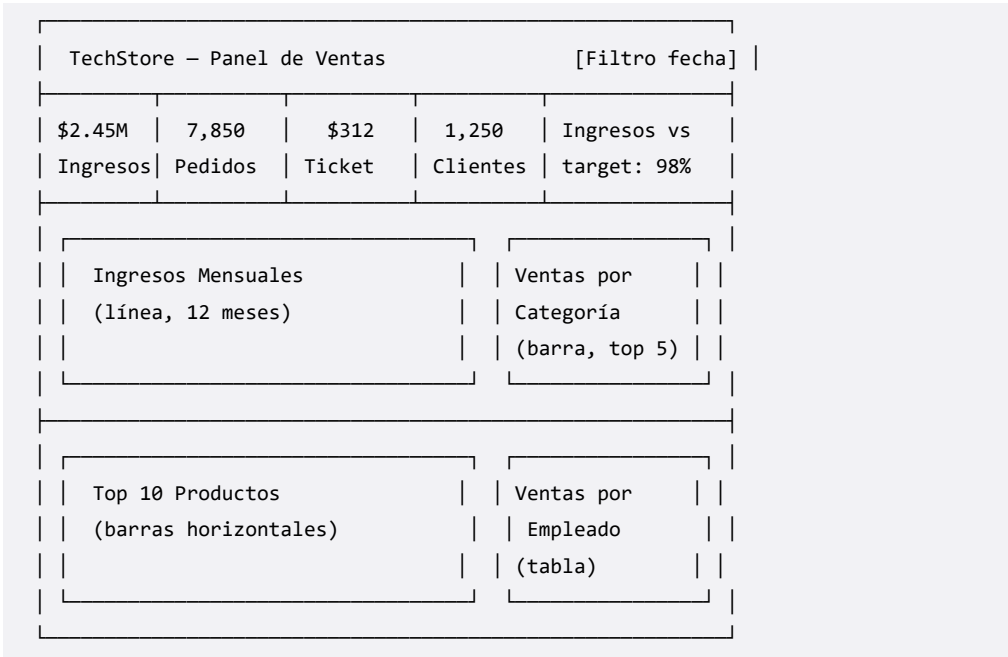
Visual	Uso	Ejemplo TechStore
Líneas	Evolución temporal	Ingresos mensuales 12 meses
Área	Volumen + tendencia	Volumen de ventas acumulado
Líneas múltiples	Comparar tendencias	Ventas por categoría en el tiempo

Detalle y análisis

Visual	Uso	Ejemplo TechStore
Tabla	Datos detallados	Top 10 productos
Matriz	Resumen por 2+ dimensiones	Ventas por categoría y mes
Mapa	Datos geográficos	Ventas por ciudad
Gráfico de dispersión	Correlación	Precio vs cantidad vendida

Diseñando el dashboard de TechStore

Página 1: Resumen Ejecutivo



Formato y estilo

Paleta de colores recomendada:

- Principal: Azul oscuro (#1B2A4A)
- Secundario: Azul medio (#2E86AB)
- Acento: Naranja (#E67E22)
- Éxito: Verde (#27AE60)
- Alerta: Rojo (#E74C3C)
- Fondo: Blanco/Gris claro (#F8F9FA)

Tipografía: Segoe UI (estándar Power BI), títulos en bold

Formato condicional

Power BI permite colorear celdas según su valor:

1. En una tabla/matriz, selecciona la columna
2. "Formato condicional" "Color de fondo"
3. Reglas:
 - Ingresos > \$100K verde
 - Ingresos entre \$50K y \$100K amarillo
 - Ingresos < \$50K rojo

Reglas de formato

```
-- Medida para formato condicional de semáforo
Semáforo Crecimiento =
VAR crecimiento = [Crecimiento %]
RETURN SWITCH(
    TRUE(),
    crecimiento >= 0.05, 1, -- Verde
    crecimiento >= 0, 2, -- Amarillo
    3 -- Rojo
)
```

Segmentaciones (Slicers)

Las segmentaciones permiten a los usuarios filtrar interactivamente:

Tipo	Cuándo usarlo
Lista	Pocos valores (categorías, empleados)
Desplegable	Muchos valores (productos, ciudades)
Control deslizante	Rangos numéricos (precio, total)
Selector de fecha	Rangos de fecha
Jerarquía	Drill-down (año → trimestre → mes)

Interacciones entre visuales

Por defecto, al seleccionar un elemento en un gráfico se filtran los demás. Puedes personalizar esto:

1. Selecciona un visual
2. "Formato" "Editar interacciones"
3. Elige para cada otro visual: **Filtrar** o **No afectar**

Drill-through y tooltips

Drill-through

Permite navegar de un resumen a un detalle:

1. Crea una página de "Detalle de Categoría"
2. Arrastra `dim\_product[category]` al panel "Drill-through"
3. Desde la página principal, haz clic derecho en una categoría "Ver detalles"

Tooltips personalizados

1. Crea una página de tooltip (tamaño: 300×150)
2. Arrastra `dim_product[category]` al panel "Tooltip"
3. En la página principal, asigna este tooltip al gráfico

Marcadores (Bookmarks)

Los marcadores guardan el estado del dashboard:

1. Configura filtros, segmentaciones, página
2. "Ver" "Panel de marcadores" "Agregar"
3. Asigna botones para cambiar entre estados

Ejercicios

1. Crea una página de resumen ejecutivo con 4 tarjetas KPI
2. Agrega un gráfico de líneas con ingresos mensuales
3. Crea un gráfico de barras con ventas por categoría
4. Agrega una segmentación de fecha (range slicer)
5. ¿Cuál es el principio más importante de Tufte para dashboards?
6. Agrega formato condicional a una tabla (por color)
7. Configura drill-through para ver detalle de una categoría
8. Crea un tooltip personalizado para el gráfico de ventas
9. Usa marcadores para alternar entre dos vistas del dashboard
10. ¿Qué tipo de gráfico usarías para mostrar la correlación entre precio y cantidad?

Capítulo 8: DAX — El Lenguaje de las Fórmulas

¿Qué es DAX?

DAX (Data Analysis Expressions) es el lenguaje de fórmulas de Power BI, Power Pivot y Analysis Services. Es similar a las fórmulas de Excel, pero diseñado para trabajar con tablas relacionales.

```
Componentes de DAX:  
- Funciones: SUM, COUNT, AVERAGE, CALCULATE  
- Operadores: +, -, *, /, &&  
- Referencias: Tablas[Columna], Medidas  
- Contexto: Filtro, Fila
```

Sintaxis básica

```
NombreMedida = FUNCIÓN(Tabla[Columna])
```

```
Ingresos Totales = SUM(fact_orders[total])  
Total Pedidos = COUNTROWS(fact_orders)  
Ticket Promedio = AVERAGE(fact_orders[total])  
Cliente Máximo Pago = MAX(fact_orders[total])  
Primer Pedido = MIN(fact_orders[order_id])
```

Contexto de filtro

El **contexto de filtro** es el concepto más importante en DAX. Determina qué filas se usan para el cálculo.

```
Cuando seleccionas "Portátiles" en un slicer:  
- El contexto de filtro incluye solo productos con category = "Portátiles"  
- SUM(fact_orders[total]) suma solo pedidos de Portátiles  
- Cada visual crea su propio contexto de filtro
```

```
-- Esta medida cambia según el contexto de filtro  
Ingresos = SUM(fact_orders[total])  
  
-- Misma medida, diferentes contextos:  
-- Sin filtros: $2.45M
```

```
-- Filtro categoría = Portátiles: $980K
-- Filtro año = 2024: $820K
```

Funciones básicas de agregación

Función	Descripción	Ejemplo
`SUM`	Suma de valores	`SUM(fact_orders[total])`
`AVERAGE`	Promedio	`AVERAGE(fact_orders[total])`
`COUNT`	Cuenta valores numéricos	`COUNT(fact_orders[total])`
`COUNTA`	Cuenta valores no vacíos	`COUNTA(dim_customer[name])`
`COUNTROWS`	Cuenta filas	`COUNTROWS(fact_orders)`
`DISTINCTCOUNT`	Cuenta valores únicos	`DISTINCTCOUNT(fact_orders[customer_sk])`
`MIN` / `MAX`	Mínimo/Máximo	`MIN(fact_orders[total])`

Funciones de conteo

```
Clientes Activos = DISTINCTCOUNT(fact_orders[customer_sk])

Pedidos con Descuento =
COUNTROWS(FILTER(fact_orders, fact_orders[total] < fact_orders[total]))

Pedidos Premium =
COUNTROWS(FILTER(fact_orders, fact_orders[total] >= 1000))
```

FILTER y CALCULATE (introducción)

`CALCULATE` cambia el contexto de filtro de una medida. Es la función más poderosa de DAX.

```
-- Ventas de Portátiles
Ventas Portátiles =
CALCULATE(
    SUM(fact_orders[total]),
    dim_product[category] = "Portátiles"
)

-- O usando FILTER (más explícito)
Ventas Portátiles v2 =
CALCULATE(
    SUM(fact_orders[total]),
    FILTER(dim_product, dim_product[category] = "Portátiles")
)
```

Funciones lógicas

```
-- IF (condición, resultado_si, resultado_no)
Margen Estimado =
SUM(fact_orders[total]) * IF(
    HASONEVALUE(dim_product[category]),
```

```

    0.35,
    0.40
)

-- SWITCH (múltiples condiciones)
Categoría Ticket =
SWITCH(
    TRUE(),
    AVERAGE(fact_orders[total]) >= 1000, "Premium",
    AVERAGE(fact_orders[total]) >= 500, "Alto",
    AVERAGE(fact_orders[total]) >= 200, "Medio",
    "Bajo"
)

```

Funciones de fecha y hora

```

-- Año actual
Ventas Año Actual =
CALCULATE(
    SUM(fact_orders[total]),
    dim_date[anio] = YEAR(TODAY())
)

-- Ventas del mes actual
Ventas Mes Actual =
CALCULATE(
    SUM(fact_orders[total]),
    dim_date[mes] = MONTH(TODAY()),
    dim_date[anio] = YEAR(TODAY())
)

```

Funciones de relación

```

-- RELATED: trae un valor de la dimensión al hecho
-- (solo se usa en columnas calculadas)
Producto Categoría = RELATED(dim_product[category])

-- RELATEDTABLE: trae datos del hecho a la dimensión
-- (solo se usa en columnas calculadas)
Pedidos del Cliente = COUNTROWS(RELATEDTABLE(fact_orders))

```

Variables en DAX

Las variables hacen las medidas más legibles y eficientes:

```

Margen Neto =
VAR ingresos = SUM(fact_orders[total])
VAR coste = ingresos * 0.6
VAR margen = ingresos - coste
VAR margen_pct = DIVIDE(margen, ingresos, 0)

```

```
RETURN margen_pct
```

```
Crecimiento vs Mes Anterior =  
VAR mes_actual = SUM(fact_orders[total])  
VAR mes_anterior =  
    CALCULATE(  
        SUM(fact_orders[total]),  
        PREVIOUSMONTH(dim_date[full_date])  
    )  
VAR variacion = mes_actual - mes_anterior  
VAR variacion_pct = DIVIDE(variacion, mes_anterior, 0)  
RETURN variacion_pct
```

DIVIDE vs división directa

```
-- MAL: división directa (error si denominador = 0)  
Margen % = SUM(fact_orders[total]) * 0.4 / SUM(fact_orders[total])  
  
-- BIEN: DIVIDE (maneja división por cero)  
Margen % = DIVIDE(SUM(fact_orders[total]) * 0.4, SUM(fact_orders[total]), 0)
```

Buenas prácticas en DAX

1. **Usa variables** para cálculos intermedios
2. **Prefiere medidas** sobre columnas calculadas (excepto para segmentaciones)
3. **Usa DIVIDE** en lugar de / (evita errores)
4. **Nombra claramente:** "Ingresos Totales", "% Crecimiento"
5. **Agrupar medidas** en carpetas lógicas
6. **Comenta fórmulas complejas** con // comentarios

Ejercicios

1. Crea una medida de "Ingresos Totales" usando SUM
2. Crea "Ticket Promedio" usando AVERAGE
3. Crea "Clientes Activos" usando DISTINCTCOUNT
4. ¿Qué es el contexto de filtro? Explica con un ejemplo
5. Crea una medida "Ventas Portátiles" usando CALCULATE
6. Usa variables para crear "Margen Neto" (ingresos - coste estimado)
7. Crea una medida "Crecimiento %" vs año anterior (pista: SAMEPERIODLASTYEAR)
8. ¿Cuál es la diferencia entre FILTER y CALCULATE?
9. Crea un SWITCH que clasifique pedidos en 3 categorías
10. ¿Por qué deberías preferir DIVIDE sobre la división directa?

Checkpoint 2: Dashboard de Ventas TechStore

Objetivo

Has aprendido Power BI Desktop, modelado de datos, visualizaciones y DAX. Ahora es momento de integrar todo: **construir un dashboard profesional de ventas para TechStore** con KPIs, gráficos interactivos y formato condicional.

Requisitos

- Power BI Desktop instalado
- Star schema de TechStore conectado (techstore_bi.db o CSVs)
- Modelo con relaciones creadas
- Medidas DAX básicas definidas

Paso 1: Modelo y relaciones

Verifica que tu modelo tenga:

- `dim\_date` conectada a `fact\_orders` por `date\_sk` (*:1)
- `dim\_customer` conectada a `fact\_orders` por `customer\_sk` (*:1)
- `dim\_employee` conectada a `fact\_orders` por `employee\_sk` (*:1)
- `dim\_product` conectada a `fact\_order\_items` por `product\_sk` (*:1)
- `fact\_orders` conectada a `fact\_order\_items` por `order\_sk` (1:*)
- Jerarquía en dim_date: Año Trimestre Mes Fecha

Paso 2: Medidas DAX

Crea estas medidas en una carpeta "KPIs":

```
-- ===== KPIs Principales =====  
  
Ingresos Totales = SUM(fact_orders[total])  
Total Pedidos = COUNTROWS(fact_orders)  
Ticket Promedio = AVERAGE(fact_orders[total])  
Clientes Activos = DISTINCTCOUNT(fact_orders[customer_sk])
```

```

Ingresos Mes Anterior =
CALCULATE(
    [Ingresos Totales],
    PREVIOUSMONTH(dim_date[full_date])
)

Crecimiento % =
DIVIDE(
    [Ingresos Totales] - [Ingresos Mes Anterior],
    [Ingresos Mes Anterior],
    0
)

% vs Target =
DIVIDE([Ingresos Totales], 150000, 0)

-- ===== KPIs de Producto =====

Unidades Vendidas = SUM(fact_order_items[quantity])
Productos Distintos = DISTINCTCOUNT(fact_order_items[product_sk])
Ingresos por Producto = SUM(fact_order_items[line_total])

-- ===== KPIs de Cliente =====

Ticket Promedio por Cliente =
DIVIDE([Ingresos Totales], [Clientes Activos], 0)

Pedidos por Cliente =
DIVIDE([Total Pedidos], [Clientes Activos], 0)

-- ===== KPIs de Tiempo =====

Ingresos YTD =
TOTALYTD([Ingresos Totales], dim_date[full_date])

Ingresos por Trimestre =
CALCULATE(
    [Ingresos Totales],
    DATESQTD(dim_date[full_date])
)

-- ===== Medidas de Formato =====

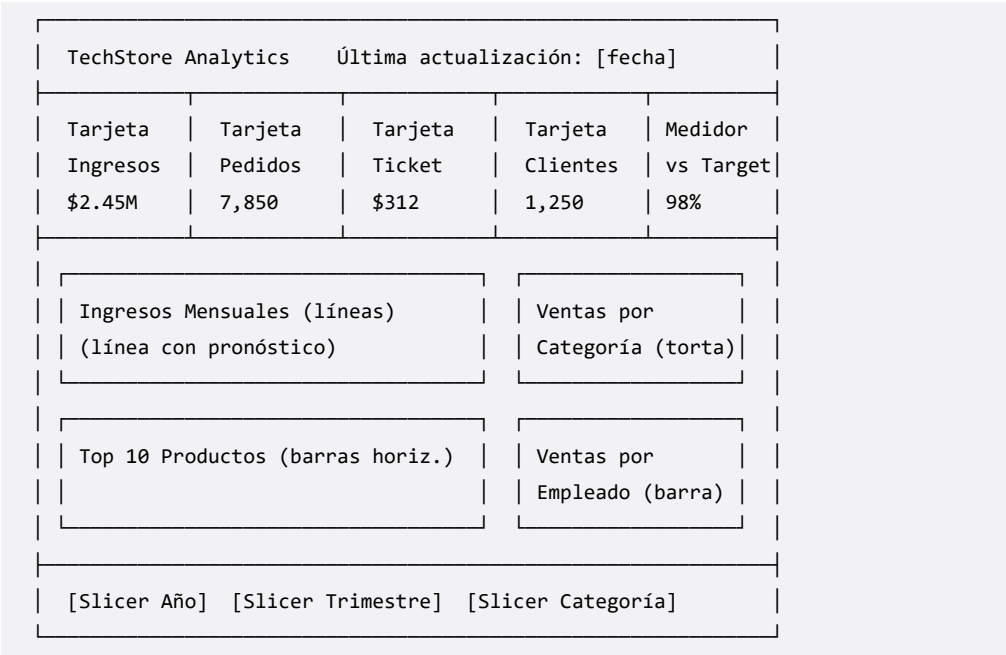
Semáforo Crecimiento =
SWITCH(
    TRUE(),
    [Crecimiento %] >= 0.05, 1,
    [Crecimiento %] >= 0, 2,
    3
)

```

Paso 3: Páginas del dashboard

Página 1: Resumen Ejecutivo

Diseño:



Página 2: Detalle de Ventas

- Matriz: Categoría × Mes (con formato condicional)
- Tabla: Top 20 productos (con barra de progreso)
- Gráfico de dispersión: Precio vs Cantidad vendida
- Mapa: Ventas por ciudad (si tienes datos de ubicación)

Página 3: KPIs por Empleado

- Gráfico de barras: Ingresos por empleado
- Tabla: métricas detalladas por empleado
- Segmentación por empleado (desplegable)
- Tarjetas: Top performer, promedio del equipo

Paso 4: Formato y estilo

1. Tema: utiliza la paleta de colores estándar
2. Títulos descriptivos en cada visual
3. Formato condicional en la matriz (colores de fondo)
4. Tooltips personalizados
5. Marca de agua o logo de TechStore

Paso 5: Interactividad

1. Todas las segmentaciones deben afectar a todos los gráficos
2. Al hacer clic en una categoría en la torta, se filtran los demás gráficos
3. Drill-through desde el Top 10 al detalle del producto
4. Botones para navegar entre páginas

Entregables del Checkpoint 2

- [] Archivo TechStore_BI.pbix con 3 páginas de dashboard
- [] Mínimo 12 medidas DAX creadas y funcionando
- [] Modelo con relaciones correctas (star schema)
- [] Formato condicional en matriz
- [] Segmentaciones interactivas
- [] Tooltips personalizados
- [] Drill-through configurado
- [] KPIs principales visibles en la página de resumen

Preguntas de reflexión

1. ¿Qué KPI crees que es más útil para la gerencia de TechStore?
2. ¿Qué filtros (slicers) son imprescindibles en un dashboard de ventas?
3. ¿Cómo mejorarías este dashboard con datos adicionales?
4. ¿Qué medida DAX fue la más compleja de implementar?

¡Felicidades! Has construido tu primer dashboard profesional en Power BI. En el Proyecto 3 profundizarás en DAX avanzado y Power Query.

Capítulo 9: DAX Avanzado — Time Intelligence y CALCULATE

CALCULATE en profundidad

`CALCULATE` es la función más importante de DAX. Cambia el contexto de filtro de una expresión.

```
CALCULATE(  
    expresión,      -- La medida o agregación a calcular  
    filtro1,        -- Nuevo filtro a aplicar  
    filtro2,        -- (Opcional) más filtros  
    ...  
)
```

CALCULATE con filtros simples

```
Ventas Portátiles =  
CALCULATE(  
    [Ingresos Totales],  
    dim_product[category] = "Portátiles"  
)  
  
Ventas 2024 =  
CALCULATE(  
    [Ingresos Totales],  
    dim_date[anio] = 2024  
)  
  
Ventas Altas =  
CALCULATE(  
    [Ingresos Totales],  
    fact_orders[total] >= 500  
)
```

CALCULATE con múltiples filtros

```
Ventas Portátiles 2024 USA =  
CALCULATE(  
    [Ingresos Totales],  
    dim_product[category] = "Portátiles",
```

```

dim_date[anio] = 2024,
dim_customer[country] = "USA"
)

```

CALCULATE + FILTER

`FILTER` da control total sobre el filtro:

```

-- Top 10% de pedidos más caros
Ventas Top10 =
CALCULATE(
    [Ingresos Totales],
    FILTER(
        fact_orders,
        fact_orders[total] >= PERCENTILE.INC(fact_orders[total], 0.9)
    )
)

```

Funciones de Time Intelligence

Acumulados

```

-- Year to Date (desde inicio del año hasta la fecha actual)
Ventas YTD =
TOTALYTD([Ingresos Totales], dim_date[full_date])

-- Quarter to Date
Ventas QTD =
TOTALQTD([Ingresos Totales], dim_date[full_date])

-- Month to Date
Ventas MTD =
TOTALMTD([Ingresos Totales], dim_date[full_date])

```

Comparación con periodos anteriores

```

-- Mes anterior
Ventas Mes Anterior =
CALCULATE(
    [Ingresos Totales],
    PREVIOUSMONTH(dim_date[full_date])
)

-- Mismo mes del año anterior
Ventas Mismo Mes Año Anterior =
CALCULATE(
    [Ingresos Totales],
    SAMEPERIODLASTYEAR(dim_date[full_date])
)

-- Fecha equivalente del periodo anterior
Ventas Periodo Anterior =

```

```

CALCULATE(
    [Ingresos Totales],
    DATEADD(dim_date[full_date], -1, MONTH)
)

```

Promedios móviles

```

-- Promedio móvil de 3 meses
Ventas Promedio 3M =
CALCULATE(
    AVERAGEX(
        DATESINPERIOD(
            dim_date[full_date],
            LASTDATE(dim_date[full_date]),
            -3,
            MONTH
        ),
        [Ingresos Totales]
    )
)

-- O más simple con DATESINPERIOD
Ventas Promedio Móvil 3M =
CALCULATE(
    [Ingresos Totales],
    DATESINPERIOD(dim_date[full_date], LASTDATE(dim_date[full_date]), -3, MONTH)
) / 3

```

Periodo vs Periodo Anterior

```

Variación vs Año Anterior =
[Ingresos Totales] - [Ventas Mismo Mes Año Anterior]

% Crecimiento Anual =
DIVIDE(
    [Variación vs Año Anterior],
    [Ventas Mismo Mes Año Anterior],
    0
)

```

Funciones de inteligencia de tiempo avanzadas

```

-- Primer y último día del mes
Primer Día Mes = STARTOFMONTH(dim_date[full_date])
Último Día Mes = ENDOFMONTH(dim_date[full_date])

-- Fechas hasta la fecha actual
Ventas Hasta Hoy =
CALCULATE(
    [Ingresos Totales],
    DATESBETWEEN(

```

```

        dim_date[full_date],
        BLANK(),
        TODAY()
    )
)

-- Próximo periodo (para pronóstico)
Ventas Próximo Mes =
CALCULATE(
    [Ingresos Totales],
    NEXTMONTH(dim_date[full_date])
)

```

Context Transition

Context Transition ocurre cuando una medida calculada en contexto de fila se evalúa en contexto de filtro. Pasa automáticamente al usar agregaciones dentro de **FILTER** o en columnas calculadas.

```

-- En columna calculada (contexto de fila)
Cada Fila = [Ingresos Totales]
-- MAL: No funciona (medida no se puede usar en contexto de fila)

Cada Fila Correcto =
CALCULATE([Ingresos Totales])
-- BIEN: CALCULATE fuerza la transición de contexto

```

ALL, ALLEXCEPT y REMOVEFILTERS

Estas funciones **eliminan** filtros del contexto.

```

-- Porcentaje del total general
% del Total =
DIVIDE(
    [Ingresos Totales],
    CALCULATE([Ingresos Totales], ALL(dim_product))
)

-- Lo mismo, pero respetando filtros de fecha
% del Total (por fecha) =
DIVIDE(
    [Ingresos Totales],
    CALCULATE([Ingresos Totales], ALLEXCEPT(dim_product, dim_date))
)

-- Cambiar el contexto de filtro
Ventas sin Filtro Fecha =
CALCULATE([Ingresos Totales], REMOVEFILTERS(dim_date))

```

Funciones de ranking

```
-- Ranking de productos por ingresos (en una tabla)
Ranking Producto =
RANKX(
    ALL(dim_product),
    [Ingresos Totales],
    ,
    DESC,
    Dense
)

-- Top N productos
Top 5 Productos =
CALCULATE(
    [Ingresos Totales],
    TOPN(5, ALL(dim_product), [Ingresos Totales])
)
```

Funciones de evaluación (ITERATE)

Estas funciones operan fila por fila:

```
-- SUMX: suma de una expresión evaluada por fila
Ingresos por Item =
SUMX(
    fact_order_items,
    fact_order_items[quantity] * fact_order_items[unit_price]
)

-- AVERAGEX: promedio de una expresión por fila
Ticket Promedio por Día =
AVERAGEX(
    VALUES(dim_date[full_date]),
    [Ingresos Totales]
)

-- MAXX / MINX: encontrar máximo/mínimo por fila
Mejor Día de Ventas =
MAXX(
    VALUES(dim_date[full_date]),
    [Ingresos Totales]
)
```

Ejercicios

1. Usa CALCULATE para crear "Ventas de Smartphones" filtrando por categoría
2. Crea una medida YTD (Year to Date) de ingresos
3. ¿Qué hace SAMEPERIODLASTYEAR? Escribe un ejemplo
4. Crea un promedio móvil de 3 meses de ingresos
5. Usa ALL para calcular el % de cada producto sobre el total
6. Crea un ranking de categorías usando RANKX
7. ¿Qué es context transition? ¿Cuándo ocurre?

8. Usa SUMX para calcular ingresos totales desde fact_order_items
9. Crea una medida que muestre el crecimiento vs el mismo mes del año anterior
10. Usa TOPN para mostrar solo los 5 productos más vendidos

Capítulo 10: Power Query — ETL y Transformación

¿Qué es Power Query?

Power Query es el motor de ETL (Extracción, Transformación y Carga) de Power BI. Te permite conectar, limpiar y transformar datos antes de cargarlos al modelo.

Fuente → Power Query (ETL) → Modelo → Visualizaciones

Características:

- Interfaz gráfica (sin código) + lenguaje M
- 1000+ conectores (SQL, Excel, CSV, Web, APIs)
- Transformaciones sin modificar la fuente original
- Repetible: actualizas datos, las transformaciones se re-aplican

El Editor de Power Query

1. En Power BI Desktop: "Transformar datos" (cinta de opciones)
2. Se abre el Editor de Power Query
3. Panel izquierdo: lista de consultas (tablas)
4. Panel central: vista previa de datos
5. Panel derecho: "Pasos aplicados" (cada transformación es un paso)

Pasos aplicados

Cada clic en Power Query crea un paso que se guarda en secuencia:

```
Consulta: fact_orders
├─ Origen (SQLite)
├─ Navegación (tabla seleccionada)
├─ Tipo cambiado (tipos de datos)
├─ Filtros aplicados (solo 2024)
├─ Columna renombrada
└─ Carga en modelo
```

Puedes reordenar, eliminar o modificar cualquier paso.

Transformaciones básicas

Limpieza de columnas

- Eliminar filas superiores/inferiores
- Eliminar duplicados
- Mantener filas (rango)
- Rellenar hacia abajo/arriba

Columnas

- Cambiar tipo: texto, número entero, decimal, fecha
- Renombrar columnas
- Duplicar columnas
- Dividir columna (por delimitador, por número de caracteres)
- Extraer: texto antes/después, primeros/últimos caracteres, rango
- Reemplazar valores

Transformación de texto

- Mayúsculas / minúsculas / título
- Quitar espacios
- Reemplazar texto
- Agregar prefijo/sufijo
- Extraer XML/JSON

Combinar consultas

Merge (JOIN)

Equivalente a JOIN en SQL:

1. "Combinar" "Combinar consultas"
2. Selecciona las tablas y columnas clave
3. Elige el tipo de JOIN:
 - Left Outer (izquierda)
 - Right Outer (derecha)
 - Full Outer (completo)
 - Inner (interno)
 - Anti (solo no coincidentes)

Append (UNION)

Equivalente a UNION ALL en SQL:

1. "Combinar" "Anexar consultas"
2. Selecciona las tablas a unir
3. Las columnas deben coincidir

Agrupación y agregación

1. "Transformar" "Agrupar por"
2. Columna de agrupación: `category`
3. Nueva columna: "Ingresos" Suma de `line\_total`

Equivalente M generado:

```
= Table.Group(
    fact_order_items,
    {"product_sk"},
    {"Ingresos", each List.Sum([line_total]), type number}}
)
```

Columnas condicionales

1. "Agregar columna" "Columna condicional"
2. Ejemplo: clasificar pedidos por monto

```
= Table.AddColumn(
    fact_orders,
    "Categoría Pedido",
    each if [total] >= 1000 then "Premium"
        else if [total] >= 500 then "Alto"
        else if [total] >= 200 then "Medio"
        else "Bajo"
)
```

Columnas personalizadas (fórmulas M)

```
// Descuento simulado del 10% para pedidos > $500
= Table.AddColumn(
    fact_orders,
    "Descuento",
    each if [total] > 500 then [total] * 0.1 else 0
)

// Nombre completo
= Table.AddColumn(
    dim_customer,
    "Nombre Completo",
    each [name]
)

// Edad del cliente
= Table.AddColumn(
    dim_customer,
    "Años Registrado",
    each Duration.Days(DateTime.Date(DateTime.LocalNow()) - [registration_date]) / 365
)
```

Funciones M útiles

```
// Fecha
Date.Year([fecha])
Date.Month([fecha])
Date.Day([fecha])
```

```

Date.DayOfWeek([fecha])
Date.QuarterOfYear([fecha])

// Texto
Text.Upper([texto])
Text.Lower([texto])
Text.Trim([texto])
Text.Length([texto])
Text.Start([texto], 5)
Text.End([texto], 3)

// Lógica
if [columna] > 10 then "Alto" else "Bajo"

```

Parámetros en Power Query

Los parámetros permiten reutilizar valores:

1. "Administrar parámetros" "Nuevo parámetro"
2. Nombre: `AñoActual`, Tipo: Número, Valor: 2024
3. Úsalo en los pasos: `if [anio] = AñoActual then ...`

ETL práctico para TechStore

Paso 1: Conectar y limpiar

1. Conecta las 6 tablas del star schema
2. En fact_orders: filtra `total > 0` (eliminar valores negativos)
3. En dim_customer: reemplazar ciudades vacías con "Sin Ciudad"
4. En dim_product: categorizar productos sin categoría como "General"

Paso 2: Crear dimensión de tiempo completa

1. Crea una nueva consulta en blanco
2. Usa la función `List.Dates` para generar el calendario

```

let
    FechaInicio = #date(2023, 1, 1),
    FechaFin = #date(2026, 12, 31),
    Fechas = List.Dates(FechaInicio, Duration.Days(FechaFin - FechaInicio) + 1, #duration(1, 0, 0, 0)),
    Tabla = Table.FromList(Fechas, Splitter.SplitByNothing(), {"full_date"}),
    #"Tipo Cambiado" = Table.TransformColumnTypes(Tabla, {"full_date", type date}),
    #"Año Agregado" = Table.AddColumn(#"Tipo Cambiado", "anio", each Date.Year([full_date])),
    #"Mes Agregado" = Table.AddColumn(#"Año Agregado", "mes", each Date.Month([full_date])),
    #"Nombre Mes" = Table.AddColumn(#"Mes Agregado", "mes_nombre", each Date.MonthName([full_date])),
    #"Trimestre" = Table.AddColumn(#"Nombre Mes", "trimestre", each Date.QuarterOfYear([full_date])),
    #"Día Semana" = Table.AddColumn(#"Trimestre", "dia_semana_num", each Date.DayOfWeek([full_date]) + 1),
    #"Nombre Día" = Table.AddColumn(#"Día Semana", "dia_semana_nombre", each Date.DayOfWeekName([full_date])),
    #"Fin Semana" = Table.AddColumn(#"Nombre Día", "es_fin_semana", each if Date.DayOfWeek([full_date]) >= 5 then true else false),
in
    #"Fin Semana"

```

Paso 3: Fusionar consultas para tabla completa

Crea una consulta que unifique orders + customer + employee:

1. "Combinar" fact_orders + dim_customer (por customer_sk)
2. Expande: name, city, country
3. "Combinar" resultado + dim_employee (por employee_sk)
4. Expande: employee name, position

Buenas prácticas en Power Query

1. **Nombra cada paso** descriptivamente (no "Paso personalizado1")
2. **Elimina pasos innecesarios** al final
3. **Usa parámetros** para valores que cambian (año, umbral)
4. **Documenta consultas complejas** con comentarios
5. **Agrupar consultas** relacionadas en carpetas
6. **Mantén la fuente original** — nunca modifiques los datos fuente

Ejercicios

1. Conecta las 6 tablas del star schema en Power Query
2. Aplica una transformación: filtra pedidos con total > 0
3. Crea una columna condicional "Categoría de Precio" en dim_product
4. ¿Qué es un paso aplicado y cómo se reordena?
5. Combina fact_orders + dim_customer usando Merge
6. ¿Cuál es la diferencia entre Merge y Append?
7. Crea la tabla dim_date completa en Power Query usando M
8. Agrupa fact_order_items por product_sk y suma line_total
9. Crea un parámetro "Año Actual" y úsalo en un filtro
10. Documenta cada paso de tu consulta principal con nombres descriptivos

Capítulo 11: Data Warehousing y Modelo Estrella en Power BI

Del star schema al modelo semántico

El modelo estrella que construiste en el Proyecto 1 es la base. En Power BI, este modelo se convierte en un **modelo semántico** que define cómo los consumidores interactúan con los datos.

Tablas de hechos: diseño fino

Granularidad correcta

Elige la granularidad adecuada para cada tabla de hechos:

Tabla	Granularidad	Medidas	Filas aprox
fact_orders	1 fila = 1 pedido	total, item_count	6,000
fact_order_items	1 fila = 1 línea de pedido	quantity, unit_price, line_tot	~15,000
fact_daily_orders	1 fila = 1 día	pedidos, ingresos	~1,100
fact_monthly_summary	1 fila = 1 mes + 1 cat	ingresos, unidades	~200

NW: no crees tablas de hechos resumidas si los datos caben en RAM. El motor de Power BI comprime muy bien los datos detallados.

Medidas vs columnas en fact tables

```
-- Columna en fact_order_items (calculada en ETL)
line_total = [quantity] * [unit_price]

-- Medida en Power BI (calculada en consulta)
Ingresos por Producto = SUMX(fact_order_items, [quantity] * [unit_price])
```

Recomendación: calcula `line\_total` en Power Query (columna) para velocidad, pero usa medidas para lógica de negocio compleja.

Tablas de dimensión: enriquecimiento

Añadir columnas derivadas

```
-- En dim_date (columnas calculadas)
Año-Mes = dim_date[anio] & "-" & FORMAT(dim_date[mes], "00")
```

```

Trimestre Nombre = "Q" & dim_date[trimestre] & " " & dim_date[anio]
Es Finde Semana = IF(dim_date[es_fin_semana] = 1, "Sí", "No")

-- En dim_customer
Segmento Cliente =
SWITCH(
    TRUE(),
    dim_customer[city] IN ("New York", "Los Angeles", "Chicago"), "Metrópolis",
    dim_customer[city] IN ("Miami", "Houston", "Phoenix"), "Gran Ciudad",
    "Otras Ciudades"
)

```

Dimensiones Role-Playing

Una misma dimensión puede cumplir múltiples roles. Ejemplo: `dim\_date` usada como fecha de pedido, fecha de envío, fecha de pago.

En Power BI: crea copias de la tabla con diferentes nombres:

```

-- Crea una copia de dim_date (sin cargar datos duplicados)
dim_date_envío = dim_date -- Role-playing dimension
dim_date_pago = dim_date -- Role-playing dimension

```

Luego relaciona cada copia con la columna correspondiente en fact_orders.

Tablas de calendario completas

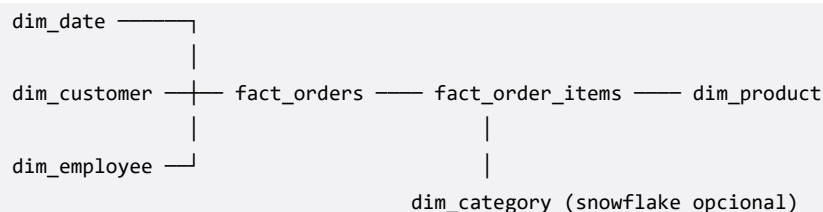
Una buena tabla de calendario debe cubrir:

```

-- Calendario fiscal (para empresas con año fiscal no calendario)
-- Ejemplo: año fiscal de TechStore empieza en Octubre
SELECT
    full_date,
    anio,
    CASE WHEN mes >= 10 THEN anio + 1 ELSE anio END AS anio_fiscal,
    CASE WHEN mes >= 10 THEN mes - 9 ELSE mes + 3 END AS mes_fiscal,
    (CASE WHEN mes >= 10 THEN (mes - 10) / 3 + 1 ELSE (mes + 2) / 3 + 1 END) AS trimestre_fiscal,
    ...
FROM dim_date;

```

Modelo de datos avanzado para TechStore



Agregar tablas auxiliares

```

-- Tabla de objetivos (importada de Excel)
Objetivos Mensuales =

```

```

UNION(
    ROW("Año", 2024, "Mes", 1, "Objetivo", 130000),
    ROW("Año", 2024, "Mes", 2, "Objetivo", 135000),
    ...
)

-- Medida: cumplimiento vs objetivo
% Cumplimiento =
DIVIDE(
    [Ingresos Totales],
    SUM(Objetivos_Mensuales[Objetivo]),
    0
)

```

Rendimiento del modelo

Estrategias de optimización

1. **Elimina columnas innecesarias** en Power Query
2. **Usa tipos de datos correctos** (entero vs decimal, fecha vs texto)
3. **Agrupar datos de alta cardinalidad** (ej: combinar ciudades pequeñas en "Otras")
4. **Deshabilita auto-date/time** (Archivo Opciones Carga de datos)
5. **Usa agregaciones precalculadas** para tablas muy grandes

Cardinalidad y compresión

Power BI comprime mejor columnas con:

- **Baja cardinalidad:** categorías, estados (pocos valores únicos)
- **Datos ordenados:** ordena por la columna de alta cardinalidad
- **Enteros vs texto:** los enteros comprimen mejor

```

// En Power Query: ordenar por clave para mejor compresión
= Table.Sort(fact_orders, {"date_sk", Order.Ascending})

```

Working with Multiple Fact Tables

A veces tienes múltiples tablas de hechos (ventas, inventario, marketing). Comparten dimensiones conformadas:

```

dim_date — fact_orders
dim_date — fact_inventory
dim_date — fact_marketing_spend

dim_product — fact_order_items
dim_product — fact_inventory

```

Las medidas de diferentes hechos se pueden combinar siempre que compartan dimensiones.

Ejercicios

1. Agrega una columna "Año-Mes" (formato "2024-01") a dim_date
2. Crea una dimensión role-playing para fecha de envío
3. ¿Qué es una tabla de calendario fiscal? ¿Cómo la crearías?
4. Importa una tabla de objetivos mensuales desde Excel
5. Crea una medida de % Cumplimiento vs Objetivo
6. ¿Cómo optimizarías el rendimiento de un modelo con 50M de filas?
7. ¿Qué columnas eliminarías de fact_orders para mejorar compresión?
8. Crea una tabla de agregación mensual (fact_monthly_summary)
9. ¿Cuál es la ventaja de las dimensiones conformadas?
10. Diseña un modelo con 2 tablas de hechos (ventas + inventario)

Capítulo 12: Medidas Avanzadas y KPIs Dinámicos

KPIs dinámicos con segmentaciones

Los KPIs dinámicos cambian según la selección del usuario. Para lograrlo, necesitas tablas de parámetros y medidas inteligentes.

Tabla de parámetros

Crea una tabla desconectada (no relacionada con el modelo):

```
Tabla KPIs =
DATATABLE(
    "KPI", STRING,
    "Orden", INTEGER,
    "Descripción", STRING,
    {
        {"Ingresos Totales", 1, "Suma de todos los ingresos"},
        {"Total Pedidos", 2, "Número total de pedidos"},
        {"Ticket Promedio", 3, "Ingreso promedio por pedido"},
        {"Clientes Activos", 4, "Clientes que compraron"},
        {"Crecimiento %", 5, "Variación vs mes anterior"}
    }
)
```

Medida genérica que cambia según selección

```
KPI Seleccionado =
SWITCH(
    SELECTEDVALUE('Tabla KPIs'[KPI]),
    "Ingresos Totales", [Ingresos Totales],
    "Total Pedidos", [Total Pedidos],
    "Ticket Promedio", [Ticket Promedio],
    "Clientes Activos", [Clientes Activos],
    "Crecimiento %", [Crecimiento %],
    BLANK()
)
```

Matriz dinámica con KPIs

Usa la tabla de parámetros como filas de una matriz y la medida genérica como valor:

```
Filas: Tabla KPIs[KPI]
Columnas: dim_date[anio], dim_date[mes]
Valores: [KPI Seleccionado]
```

El usuario ve todos los KPIs en una sola matriz sin necesidad de múltiples gráficos.

Segmentación de fecha dinámica

```
Tabla Periodos =
DATATABLE(
    "Periodo", STRING,
    "Días", INTEGER,
    {
        {"Últimos 7 días", -7},
        {"Últimos 30 días", -30},
        {"Últimos 90 días", -90},
        {"Últimos 12 meses", -365},
        {"Todo el histórico", -9999}
    }
)
```

```
Ventas Periodo Dinámico =
VAR dias = SELECTEDVALUE('Tabla Periodos'[Días], -30)
RETURN
CALCULATE(
    [Ingresos Totales],
    DATESINPERIOD(
        dim_date[full_date],
        LASTDATE(dim_date[full_date]),
        dias,
        DAY
    )
)
```

KPIs con formato condicional avanzado

```
-- Medida que devuelve el formato del semáforo
Formato KPI =
SWITCH(
    TRUE(),
    [Crecimiento %] >= 0.05, "verde",
    [Crecimiento %] >= 0, "amarillo",
    "rojo"
)

-- En formato condicional, usas reglas basadas en campo
-- 1. Selecciona la medida
-- 2. Formato condicional → Color de fondo → Formato por: "Formato KPI"
```

Forecasting con DAX

Power BI no tiene forecasting nativo en DAX, pero puedes simular tendencias:

```
-- Tendencia lineal simple
Pronóstico Próximo Mes =
VAR ultimo_mes_con_datos = LASTNONBLANK(dim_date[full_date], [Ingresos Totales])
VAR ultimo_valor = CALCULATE([Ingresos Totales], ultimo_mes_con_datos)
VAR promedio_ultimos_3 =
    CALCULATE(
        AVERAGEX(
            DATESINPERIOD(dim_date[full_date], ultimo_mes_con_datos, -3, MONTH),
            [Ingresos Totales]
        )
    )
VAR tendencia = ultimo_valor - promedio_ultimos_3
RETURN ultimo_valor + tendencia
```

What-If Analysis

Power BI admite análisis What-If con parámetros:

1. "Modelado" "Nuevo parámetro"
2. Nombre: "Descuento %", Tipo: Decimal, Rango: 0 a 0.5, Incremento: 0.05
3. Se crea una tabla `Descuento` y una medida `Descuento[Valor]`

```
Ingresos con Descuento =
[Ingresos Totales] * (1 - [Valor Descuento])

Impacto Descuento =
[Ingresos Totales] - [Ingresos con Descuento]
```

Agrega un slicer con el parámetro para que el usuario ajuste el descuento.

Medidas de Pareto (80/20)

```
% Acumulado Producto =
VAR total_general = CALCULATE([Ingresos Totales], ALL(dim_product))
VAR ingresos_producto = [Ingresos Totales]
VAR productos_mayores =
    FILTER(
        ALL(dim_product),
        [Ingresos Totales] >= ingresos_producto
    )
VAR suma_mayores =
    CALCULATE([Ingresos Totales], productos_mayores)
RETURN DIVIDE(suma_mayores, total_general, 0)

Es Pareto 80 =
IF([% Acumulado Producto] <= 0.8, "80%", "20%")
```

Medidas de cohortes

```
-- Primera compra del cliente
```

```

Primera Compra Cliente =
CALCULATE(
    MIN(dim_date[full_date]),
    ALLEXCEPT(dim_customer, dim_customer[customer_sk])
)

-- Mes de cohorte (para análisis de cohortes)
Mes Cohorte = FORMAT([Primera Compra Cliente], "YYYY-MM")

```

Tarjetas con indicadores

```

Tarjeta Ingresos con Indicador =
VAR ingresos_actuales = [Ingresos Totales]
VAR ingresos_anteriores = [Ingresos Mes Anterior]
VAR variacion = ingresos_actuales - ingresos_anteriores
VAR icono = IF(variacion >= 0, UNICHAR(9650), UNICHAR(9660)) -- ▲ o ▼
VAR color = IF(variacion >= 0, "verde", "rojo")
RETURN
    ingresos_actuales & " " & icono & " " & FORMAT(variacion, "$#,##0")

```

Biblioteca de medidas útiles

```

-- ===== Medidas para cualquier modelo BI =====

% Penetración =
DIVIDE(
    DISTINCTCOUNT(fact_orders[customer_sk]),
    COUNTROWS(dim_customer),
    0
)

Frecuencia Compra =
DIVIDE(
    [Total Pedidos],
    [Clientes Activos],
    0
)

Ticket por Cliente =
DIVIDE(
    [Ingresos Totales],
    [Clientes Activos],
    0
)

Días Desde Última Venta =
VAR ultima_venta =
    CALCULATE(
        MAX(dim_date[full_date]),
        ALL(dim_date)
    )

```

```

    )
    RETURN DATEDIFF(ultima_venta, TODAY(), DAY)

    Ingreso Promedio por Día =
    DIVIDE(
        [Ingresos Totales],
        COUNTROWS(dim_date),
        0
    )

```

Ejercicios

1. Crea una tabla de parámetros "KPIs" con 5 KPIs seleccionables
2. Crea una medida "KPI Seleccionado" que cambie según el parámetro
3. Implementa un análisis What-If con un parámetro de descuento
4. ¿Cómo harías un análisis de Pareto (80/20) en DAX?
5. Crea un indicador de tarjeta con / y color según tendencia
6. Crea una medida de cohorte (primera compra por cliente)
7. ¿Qué es una tabla desconectada? ¿Para qué sirve?
8. Implementa una segmentación de fecha dinámica (últimos 7, 30, 90 días)
9. Crea una medida de "Frecuencia de Compra" (pedidos / cliente)
10. Diseña una matriz dinámica que muestre todos los KPIs por mes

Checkpoint 3: Modelo Semántico y KPIs Avanzados

Objetivo

Has aprendido DAX avanzado, Power Query y modelado de datos. Ahora es momento de llevar tu modelo Power BI al siguiente nivel: **crear un modelo semántico profesional con KPIs dinámicos, análisis What-If y segmentaciones inteligentes.**

Requisitos

- Archivo TechStore_BI.pbix del Checkpoint 2
- Modelo star schema conectado y validado
- Medidas DAX básicas funcionando

Paso 1: Tablas auxiliares (desconectadas)

Crea estas tablas usando "Introducir datos" en Power BI:

Tabla KPIs

```
Tabla KPIs =
DATATABLE(
    "KPI", STRING,
    "Orden", INTEGER,
    "Descripción", STRING,
    {
        {"Ingresos Totales", 1, "Suma de ingresos"},
        {"Total Pedidos", 2, "Número de pedidos"},
        {"Ticket Promedio", 3, "Ingreso promedio por pedido"},
        {"Clientes Activos", 4, "Clientes únicos que compraron"},
        {"Crecimiento %", 5, "Variación vs mes anterior"}
    }
)
```

Tabla Periodos

```
Tabla Periodos =
DATATABLE(
    "Periodo", STRING,
```

```

"Días", INTEGER,
"Orden", INTEGER,
{
    {"Últimos 7 días", -7, 1},
    {"Últimos 30 días", -30, 2},
    {"Últimos 90 días", -90, 3},
    {"Últimos 12 meses", -365, 4},
    {"Todo el histórico", -9999, 5}
}
)

```

Tabla Objetivos (simulada)

```

Objetivos Mensuales =
DATATABLE(
    "Año", INTEGER,
    "Mes", INTEGER,
    "Objetivo Ingresos", CURRENCY,
    "Objetivo Pedidos", INTEGER,
    {
        {2024, 1, 130000, 400},
        {2024, 2, 135000, 410},
        {2024, 3, 140000, 420},
        {2024, 4, 145000, 430},
        {2024, 5, 150000, 440},
        {2024, 6, 155000, 450},
        {2024, 7, 160000, 460},
        {2024, 8, 165000, 470},
        {2024, 9, 170000, 480},
        {2024, 10, 175000, 490},
        {2024, 11, 180000, 500},
        {2024, 12, 200000, 550}
    }
)

```

Paso 2: Medidas avanzadas

Agrega estas medidas a tu modelo:

```

-- ===== KPIs Dinámicos =====

KPI Seleccionado =
SWITCH(
    SELECTEDVALUE('Tabla KPIs'[KPI]),
    "Ingresos Totales", [Ingresos Totales],
    "Total Pedidos", [Total Pedidos],
    "Ticket Promedio", [Ticket Promedio],
    "Clientes Activos", [Clientes Activos],
    "Crecimiento %", [Crecimiento %],
    BLANK()
)

```

```

Ventas Periodo Dinámico =
VAR dias = SELECTEDVALUE('Tabla Periodos'[Días], -30)
RETURN
CALCULATE(
    [Ingresos Totales],
    DATESINPERIOD(
        dim_date[full_date],
        LASTDATE(dim_date[full_date]),
        dias,
        DAY
    )
)

-- ===== Cumplimiento vs Objetivo =====

% Cumplimiento Ingresos =
DIVIDE(
    [Ingresos Totales],
    SUM(Objetivos_Mensuales[Objetivo Ingresos]),
    0
)

% Cumplimiento Pedidos =
DIVIDE(
    [Total Pedidos] * 1.0,
    SUM(Objetivos_Mensuales[Objetivo Pedidos]),
    0
)

-- ===== Análisis What-If =====

Ingresos con Descuento =
[Ingresos Totales] * (1 - [Valor Descuento])

Impacto Descuento =
[Ingresos Totales] - [Ingresos con Descuento]

-- ===== Pareto =====

% Acumulado =
VAR total_general = CALCULATE([Ingresos Totales], ALL(dim_product))
VAR ingresos_actual = [Ingresos Totales]
VAR productos_superiores =
    FILTER(
        ALL(dim_product),
        [Ingresos Totales] >= ingresos_actual
    )
RETURN
DIVIDE(
    CALCULATE([Ingresos Totales], productos_superiores),
    total_general
)

```

```

Es Pareto = IF([% Acumulado] <= 0.8, "80%", "20%")

-- ===== Indicadores de Tarjeta =====

Variación vs Mes Anterior =
[Ingresos Totales] - [Ingresos Mes Anterior]

Tendencia =
IF([Variación vs Mes Anterior] >= 0, "▲", "▼")

Color Tendencia =
IF([Variación vs Mes Anterior] >= 0, "verde", "rojo")

```

Paso 3: Páginas nuevas

Página 4: KPIs Dinámicos

- Un slicer con la tabla "Tabla KPIs" (lista)
- Una tarjeta que muestre "KPI Seleccionado"
- Matriz: Tabla KPIs[KPI] en filas, dim_date[anio] y [mes] en columnas

Página 5: What-If

- Slicer del parámetro "Descuento" (control deslizante)
- Tarjetas: Ingresos Originales, Ingresos con Descuento, Impacto
- Gráfico de líneas: comparando ingresos con/sin descuento por mes

Página 6: Análisis Avanzado

- Gráfico de Pareto (barras + línea acumulada)
- Matriz con formato condicional por semáforo
- Tabla de cohortes (opcional)

Paso 4: Formato y estilo avanzado

1. Formato condicional en matriz (color de fondo según valor)
2. Iconos en tarjetas (/ según tendencia)
3. Tooltips personalizados para KPIs
4. Marcadores para cambiar entre vistas de KPIs

Entregables del Checkpoint 3

- [] Tablas auxiliares creadas (KPIs, Periodos, Objetivos)
- [] Medida "KPI Seleccionado" funcionando con parámetro
- [] Análisis What-If con parámetro de descuento
- [] Matriz dinámica con KPIs seleccionables
- [] % Cumplimiento vs Objetivo
- [] Análisis de Pareto (80/20)

- [] Tarjetas con indicadores de tendencia
- [] Página de KPIs dinámicos
- [] Página de What-If

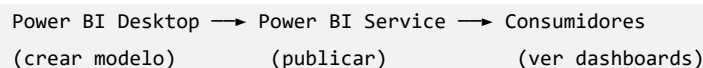
Preguntas de reflexión

1. ¿Qué ventaja tiene un KPI dinámico frente a tarjetas estáticas?
 2. ¿Cómo usarías el What-If Analysis en una reunión con la gerencia?
 3. ¿Qué insights te da el análisis de Pareto para TechStore?
 4. ¿Qué otros parámetros What-If serían útiles para TechStore?
- ¡Felicidades! Tu modelo semántico es ahora profesional y flexible. En el Proyecto 4 lo publicarás en Power BI Service para compartirlo con tu organización.

Capítulo 13: Power BI Service — Publicación y Colaboración

¿Qué es Power BI Service?

Power BI Service es la plataforma cloud donde publicas, compartes y colaboras con tus dashboards.



Planes de Power BI Service

Plan	Coste	Características
Gratuito	\$0	Solo ver contenido propio. No compartir
Pro	~\$10 USD/usuario/mes	Compartir, colaborar, workspaces
Premium Per User	~\$20 USD/usuario/mes	AI, paginados, mayores límites
Premium Capacity	Desde \$5K USD/mes	Dedicado, sin licencias por usuario

Para este libro, usaremos la **versión gratuita** para publicar y la **versión Pro** para compartir.

Publicar desde Power BI Desktop

1. En Power BI Desktop: "Publicar" (cinta Inicio)
2. Inicia sesión con tu cuenta Microsoft (gratuita)
3. Selecciona el espacio de trabajo:
 - "Mi área de trabajo" (personal, gratuito)
 - Un workspace compartido (requiere Pro)
4. Haz clic en "Seleccionar"

El informe se publica como:

- **Informe:** las visualizaciones con filtros interactivos
- **Conjunto de datos:** el modelo de datos (tablas, medidas, relaciones)
- **Panel:** (opcional) puedes crear un panel con KPIs fijos

Workspaces

Los **workspaces** son contenedores lógicos para organizar informes:

```
Workspace: "TechStore BI"
├─ Informe: Resumen Ejecutivo
├─ Informe: Ventas por Producto
├─ Informe: KPIs Gerenciales
├─ Conjunto de datos: TechStore Modelo
└─ Panel: KPIs TechStore
```

Crear un workspace

1. En Power BI Service: "Workspaces" "Crear workspace"
2. Nombre: "TechStore Analytics"
3. Descripción: "Dashboards de ventas TechStore"
4. Configura:
 - **Tipo de workspace:** Workspace clásico o basado en app
 - **Contacto:** tu email
5. "Guardar"

Roles en workspaces

Rol	Permisos
Admin	Todo: editar, compartir, agregar miembros
Member	Publicar, editar, compartir contenido
Contributor	Publicar y editar contenido
Viewer	Solo ver contenido

Compartir informes

Opción 1: Compartir directamente

1. Abre el informe en Power BI Service
2. "Compartir" (arriba a la derecha)
3. Ingresa emails de los destinatarios
4. Permisos: "Permitir que los destinatarios compartan" (opcional)
5. "Compartir"

Opción 2: Publicar en toda la organización

1. "Archivo" "Publicar en la web" (público — cuidado con datos sensibles)
2. O "Incrustar" en SharePoint o sitio web corporativo

Opción 3: Apps de Power BI

1. En el workspace, "Crear app"
2. Selecciona los informes y paneles a incluir
3. Configura navegación, colores, logo
4. Publica la app para los usuarios

Programación de actualización

Los datos locales (SQLite, Excel) no se actualizan automáticamente en la nube. Necesitas:

Gateway de datos (on-premises)

1. Descarga e instala **Power BI Gateway** (modo personal)
2. Configura la conexión a tu archivo SQLite/CSV
3. En Power BI Service:
 - Configuración del conjunto de datos "Gateway"
 - Conecta al gateway instalado
 - Programa actualización: diaria, horaria

Gateway para SQLite

Power BI Service no tiene conector nativo a SQLite. Alternativas:

1. Migra datos a BigQuery (ya lo hiciste en Libro 2)
2. Usa Excel como intermediario (exporta SQLite Excel Power BI)
3. Configura un pipeline Python que actualice CSVs y Power BI los lea

Programar actualización

1. En Power BI Service: "Conjuntos de datos" tu dataset
2. "Actualizar programado"
3. Frecuencia: Diario
4. Zona horaria: (tu zona)
5. Hora: 5:00 AM

Suscripciones a informes

Power BI permite enviar informes por email automáticamente:

1. Abre el informe
2. "Suscribirse" "Agregar suscripción"
3. Configura:
 - Frecuencia: Diaria, semanal, mensual
 - Hora de envío
 - Asunto del email
 - Destinatarios
4. "Guardar"

Los suscriptores reciben el informe como PDF adjunto.

Métricas de uso

Power BI Service proporciona analytics de uso:

1. "Configuración" "Métricas de uso"
2. Verás:
 - Vistas por informe
 - Usuarios únicos
 - Compartidos
 - Tendencias de uso

Ejercicios

1. Publica tu informe TechStore_BI.pbix en Power BI Service

2. Crea un workspace llamado "TechStore Analytics"
3. Comparte tu informe con un compañero (o con tu otro email)
4. ¿Cuál es la diferencia entre Informe y Panel en Power BI Service?
5. Programa una actualización diaria de tu conjunto de datos
6. Crea una suscripción semanal para recibir el informe por email
7. ¿Qué es Power BI Gateway y cuándo se necesita?
8. ¿Qué plan de Power BI necesitas para compartir informes?
9. Explora las métricas de uso de tu informe publicado
10. ¿Cómo manejarías datos sensibles al publicar en la web?

Capítulo 14: Seguridad — RLS, Permisos y Gobernanza

Seguridad en Power BI

La seguridad en Power BI opera en 3 capas:

```
Capa 1: Conjunto de datos (RLS)
└─ Filtros a nivel de fila según el usuario
Capa 2: Workspace y App
└─ Quién puede ver/editar/publicar
Capa 3: Fuente de datos
└─ Credenciales, gateways, cifrado
```

RLS (Row-Level Security)

RLS permite restringir qué datos ve cada usuario. Ejemplo:

- Vendedor: solo ve sus propias ventas
- Gerente regional: solo ve su región
- CEO: ve todos los datos

Paso 1: Crear roles en Power BI Desktop

1. "Modelado" "Administrar roles"
2. "Crear"
3. Nombre: "Vendedor"
4. Agrega filtro DAX:

```
-- Rol: Vendedor (solo ve sus propias ventas)
[Employee Email] = USERNAME()
-- Nota: USERNAME() devuelve el email del usuario logueado
```

Paso 2: Filtros RLS por dimensión

```
-- Rol: Gerente Regional (solo ve su región)
[Region] = USERPRINCIPALNAME()
-- Requiere tabla de usuarios con región asignada

-- Rol: Finanzas (ve todos los datos financieros)
TRUE -- Sin restricción
```

```
-- Rol: Ejecutivo (ve todo)
TRUE
```

RLS con tabla de seguridad

Para RLS avanzado, necesitas una tabla de seguridad:

```
-- Tabla de seguridad (no relacionada)
Seguridad =
DATATABLE(
    "Email", STRING,
    "Rol", STRING,
    "Región", STRING,
    {
        {"ceo@techstore.com", "Ejecutivo", "Todas"},
        {"ventas.norte@techstore.com", "Vendedor", "Norte"},
        {"ventas.sur@techstore.com", "Vendedor", "Sur"},
        {"finanzas@techstore.com", "Finanzas", "Todas"}
    }
)
```

```
-- Rol: Vendedor por región
VAR user_email = USERPRINCIPALNAME()
VAR user_region =
    CALCULATE(
        VALUES(Seguridad[Región]),
        Seguridad[Email] = user_email
    )
RETURN
dim_customer[region] = user_region
```

Probar RLS en Desktop

1. "Modelado" "Ver como"
2. Selecciona el rol "Vendedor"
3. Verifica que solo ves datos filtrados

Paso 3: Publicar y asignar roles

1. Publica el informe con RLS a Power BI Service
2. "Conjunto de datos" "Seguridad a nivel de fila"
3. Agrega miembros a cada rol:
 - Rol "Vendedor": agrega emails de vendedores
 - Rol "Ejecutivo": agrega emails de directivos
4. "Guardar"

Tabla de dimensiones de seguridad

Para RLS por región, producto o cliente:

```
-- Rol: Visibilidad por región
VAR usuario_actual = USERPRINCIPALNAME()
```

```

VAR regiones_permitidas =
    CALCULATETABLE(
        VALUES(Seguridad[Región]),
        Seguridad[Email] = usuario_actual
    )
VAR resultado =
    COUNTROWS(
        INTERSECT(
            VALUES(dim_customer[region]),
            regiones_permitidas
        )
    ) > 0
RETURN resultado

```

Permisos de workspace

Asignar roles en workspace

1. "Workspace" "Acceso"
2. Agrega usuarios:
 - **Admin:** arquitectos BI
 - **Member:** analistas que crean contenido
 - **Viewer:** consumidores de dashboards

Buenas prácticas de permisos

1. **Mínimo privilegio:** asigna solo el acceso necesario
2. **Grupos > individuos:** usa grupos de seguridad Azure AD
3. **Viewer para consumidores:** no necesitan editar
4. **App en lugar de compartir:** más control, actualización centralizada

Gobernanza en BI

La gobernanza asegura que los datos sean confiables, seguros y bien gestionados.

Principios de gobernanza

Principio	Descripción	Práctica
Calidad	Datos precisos y completos	Validaciones automáticas
Linaje	Origen y transformación de datos	Documentación de cada paso
Acceso	Solo usuarios autorizados	RLS, roles, permisos
Consistencia	Mismas definiciones en toda la org	Glosario de términos
Actualidad	Datos actualizados	Programación de refresco

Estándares para TechStore

```

-- Glosario de medidas (documentar cada una)
Medida: Ingresos Totales
Definición: Suma del campo total en fact_orders

```

Fórmula: SUM(fact_orders[total])

Frecuencia: Diaria

Responsable: Analista BI

Medida: Ticket Promedio

Definición: Ingreso promedio por pedido

Fórmula: AVERAGE(fact_orders[total])

Frecuencia: Diaria

Responsable: Analista BI

Certificación de datasets

Power BI Service permite **certificar** datasets como oficiales:

1. "Configuración" "Conjunto de datos" "Certificar"
2. Solo admins pueden certificar
3. Los datasets certificados tienen un ícono de verificación

Auditoría

Power BI Service registra actividad:

Consumo de informes: quién vio qué y cuándo

Actualizaciones: éxito/fallo, duración, datos cargados

Cambios en permisos: quién agregó/quitó usuarios

Para auditoría avanzada: **Microsoft 365 Compliance Center**.

Ejercicios

1. Crea un rol RLS "Vendedor" que filtre por empleado
2. Prueba el rol en Power BI Desktop con "Ver como"
3. Publica el informe con RLS y asigna usuarios
4. Crea una tabla de seguridad con emails y roles
5. ¿Qué diferencia hay entre RLS y OLS (Object-Level Security)?
6. Crea un rol "Gerente Regional" que filtre por ciudad
7. ¿Qué es un workspace y qué rol asignarías a un analista?
8. Documenta 3 medidas de tu modelo con nombre, fórmula y responsable
9. Configura un dataset certificado en Power BI Service
10. ¿Cómo auditarías quién vio un informe la semana pasada?

Capítulo 15: Proyecto Final — Sistema de BI Completo

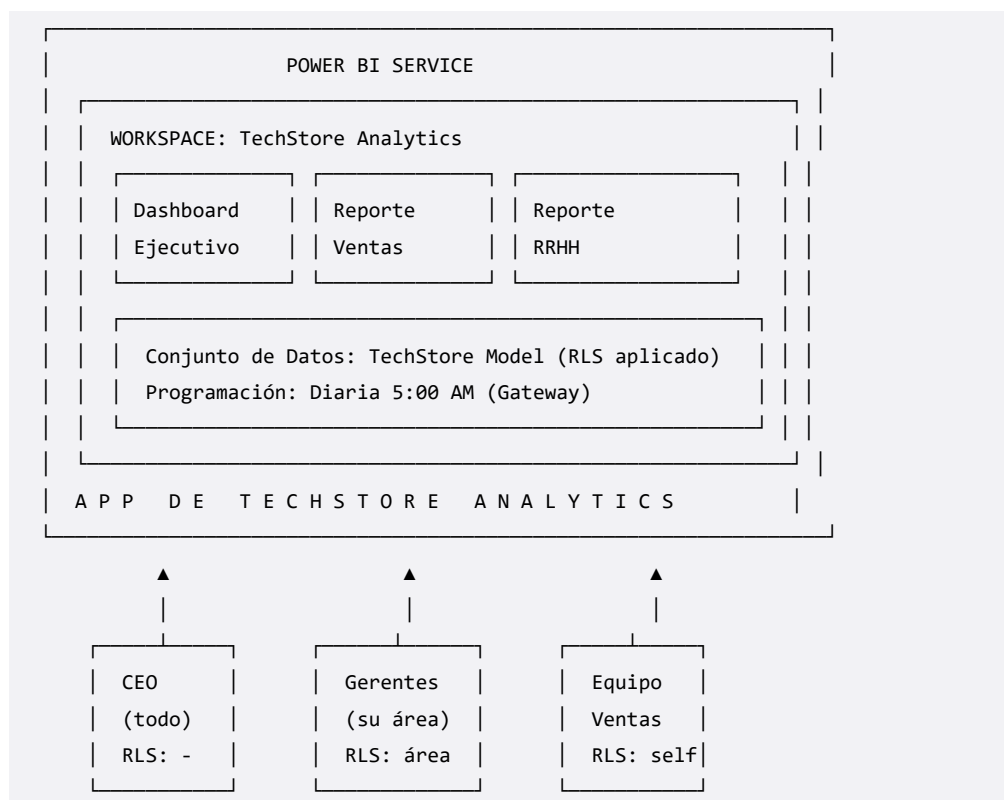
El sistema completo

En los proyectos anteriores construiste:

1. **Modelo dimensional** (star schema)
2. **Dashboard interactivo** en Power BI Desktop
3. **Medidas DAX avanzadas** y KPIs dinámicos
4. **RLS** y seguridad

Ahora es momento de integrarlo todo en un **Sistema de BI Corporativo completo** para TechStore.

Arquitectura del sistema



Paso 1: Refinar el modelo final

Asegúrate de que tu modelo tenga:

Modelo completo

```
-- Tablas del modelo
Dimensiones: dim_date, dim_customer, dim_product, dim_employee
Hechos: fact_orders, fact_order_items
Auxiliares: Tabla KPIs, Tabla Periodos, Objetivos Mensuales, Seguridad

-- Relaciones
dim_date[date_sk] → fact_orders[date_sk] (*:1)
dim_customer[customer_sk] → fact_orders[customer_sk] (*:1)
dim_employee[employee_sk] → fact_orders[employee_sk] (*:1)
dim_product[product_sk] → fact_order_items[product_sk] (*:1)
fact_orders[order_sk] → fact_order_items[order_sk] (1:*)
```

Biblioteca de medidas completa

```
-- ===== PROYECTO 1: Medidas Base =====
Ingresos Totales = SUM(fact_orders[total])
Total Pedidos = COUNTROWS(fact_orders)
Ticket Promedio = AVERAGE(fact_orders[total])
Clientes Activos = DISTINCTCOUNT(fact_orders[customer_sk])

-- ===== PROYECTO 2: Medidas de Tiempo =====
Ventas YTD = TOTALYTD([Ingresos Totales], dim_date[full_date])
Crecimiento % = DIVIDE([Ingresos Totales] - [Ingresos Mes Anterior], [Ingresos Mes Anterior], 0)
% vs Objetivo = DIVIDE([Ingresos Totales], SUM(Objetivos_Mensuales[Objetivo Ingresos]), 0)

-- ===== PROYECTO 3: Medidas Avanzadas =====
KPI Dinámico = SWITCH(...)
% Acumulado Pareto = ...
Ingresos con Descuento = ...

-- ===== PROYECTO 4: Seguridad =====
Rol Vendedor: [Vendedor Email] = USERPRINCIPALNAME()
```

Paso 2: Crear la app de Power BI

La **App** es la puerta de entrada para los usuarios finales:

1. En el workspace "TechStore Analytics", crea una app
2. Configuración:
 - **Nombre:** "TechStore Analytics"
 - **Descripción:** "Sistema de BI Corporativo — Ventas, Clientes y KPIs"
 - **Logo:** logo de TechStore (opcional)
3. **Navegación:** incluye las páginas del dashboard como secciones
4. **Permisos:** "Toda la organización" o grupos específicos

5. "Publicar app"

Paso 3: Configurar actualización automática

Para que el sistema funcione sin intervención:

Opción 1: Gateway + SQLite (local)

1. Instala Power BI Gateway (modo personal)
2. Conecta la fuente SQLite
3. Programa actualización diaria

Opción 2: Datos en BigQuery

1. Migra el star schema a BigQuery (como en Libro 2)
2. Conecta Power BI a BigQuery (DirectQuery o Import)
3. Programa actualización en BigQuery (Scheduled Queries)

Opción 3: Pipeline Python

1. Script Python que actualiza el star schema
2. Exporta a CSV
3. Power BI lee los CSVs en cada actualización

Paso 4: Documentación del sistema

Ficha técnica del sistema

```
# Sistema de BI TechStore

## Información General
- Sistema: TechStore Analytics
- Versión: 1.0
- Fecha: 2026
- Responsable: [Tu nombre]

## Componentes
- Fuente: SQLite (techstore_bi.db)
- ETL: Power Query (6 consultas)
- Modelo: Star Schema (6 tablas)
- Medidas: 25+ medidas DAX
- Dashboards: 1 app con 6 páginas
- Seguridad: RLS por rol (Vendedor, Gerente, Ejecutivo)
- Actualización: Diaria 5:00 AM (Gateway)

## KPIs Principales
1. Ingresos Totales – SUM(fact_orders[total])
2. Ticket Promedio – AVERAGE(fact_orders[total])
3. % Crecimiento – vs mes anterior
4. % Cumplimiento – vs objetivo mensual
5. Clientes Activos – DISTINCTCOUNT(customer_sk)
```

Glosario de términos

Término	Definición
Ingresos Totales	Suma de todos los pedidos confirmados
Ticket Promedio	Ingreso promedio por pedido
% Crecimiento	Variación porcentual vs mes anterior
Clientes Activos	Clientes con al menos 1 compra en el período

Paso 5: Capacitación de usuarios

Prepara una guía rápida para los consumidores del dashboard:

Cómo usar el dashboard

1. **Navegación:** usa las pestañas en la parte inferior
2. **Filtros:** selecciona año, mes, categoría en los slicers
3. **Drill-down:** haz clic en un dato para ver detalle
4. **Exportar:** haz clic derecho en un gráfico "Exportar datos"

Preguntas frecuentes

Q: ¿Cada cuánto se actualizan los datos?

R: Diariamente a las 5:00 AM

Q: ¿Puedo ver datos de mi región?

R: Sí, si eres gerente regional, el RLS filtra automáticamente

Q: ¿Qué hago si veo un dato incorrecto?

R: Contacta al equipo de BI: bi@techstore.com

Ejercicios

1. Refina tu modelo final con todas las tablas y relaciones correctas
2. Crea una app en Power BI Service con los dashboards
3. Escribe la ficha técnica completa del sistema
4. Documenta 10 medidas con nombre, fórmula y descripción
5. Prepara una guía rápida para usuarios finales
6. Configura la actualización automática del conjunto de datos
7. ¿Qué canal usarías para recibir feedback de los usuarios?
8. Diseña un plan de mantenimiento mensual del sistema
9. ¿Cómo escalarías este sistema a 100 usuarios?
10. Prepara una presentación ejecutiva de 5 slides sobre el sistema

Checkpoint 4: Portal BI Corporativo

Objetivo

Has llegado al final del libro. Este checkpoint integra todo: **publicar un portal BI corporativo completo** con Power BI Service, RLS, actualización automática y documentación.

Requisitos

- Power BI Desktop con el modelo completo
- Cuenta de Power BI Service (gratuita o Pro)
- Modelo con medidas de Proyectos 1-3

Paso 1: Publicar el informe final

1. Abre tu archivo TechStore_BI.pbix final
2. Verifica:
 - [] Todas las relaciones son correctas
 - [] Todas las medidas funcionan
 - [] Los 3 roles RLS están creados
 - [] Tablas auxiliares creadas (KPIs, Periodos, Seguridad)
3. Publica en Power BI Service

Paso 2: Configurar RLS en el servicio

1. "Conjunto de datos" tu modelo
2. "Seguridad a nivel de fila"
3. Asigna miembros a cada rol:
 - **Vendedor:** ventas@techstore.com, vendedor1@techstore.com
 - **Gerente:** gerente@techstore.com
 - **Ejecutivo:** ceo@techstore.com
4. "Guardar"
5. Prueba con "Ver como" para cada rol

Paso 3: Crear el workspace

1. Crea workspace: "TechStore BI Corporativo"

2. Agrega miembros (simulado):
 - Tú: Admin
 - Analistas: Member
 - Gerentes: Viewer
3. Mueve el informe publicado a este workspace

Paso 4: Crear la app

1. "Crear app" en el workspace
2. Nombre: "TechStore Analytics"
3. Descripción: "Portal de Business Intelligence de TechStore"
4. Páginas incluidas: todas
5. Tema: colores corporativos
6. "Publicar app"

Paso 5: Configurar actualización

1. "Conjunto de datos" "Actualizar programado"
2. Frecuencia: Diaria
3. Hora: 5:00 AM
4. Zona horaria: (tu zona)
5. Enviar notificación de fallo a tu email

Paso 6: Documentar el sistema

Completa la ficha técnica:

```
# Sistema de BI Corporativo – TechStore

## Arquitectura
- Fuente: SQLite (techstore_bi.db) / Power Query
- Modelo: Star Schema (6 tablas, 25+ medidas DAX)
- Publicación: Power BI Service (workspace corporativo)
- Seguridad: RLS (3 roles)
- Actualización: Diaria 5:00 AM

## Contenido
- 6 páginas de dashboard
- KPIs dinámicos con tabla de parámetros
- Análisis What-If (descuento)
- Matriz de Pareto (80/20)
- Cumplimiento vs objetivos

## Roles de Seguridad
- Ejecutivo: acceso total
- Gerente: filtra por región
- Vendedor: solo sus ventas

## KPIs Principales
1. Ingresos Totales: $2.45M
```

- 2. Ticket Promedio: \$312
- 3. Clientes Activos: 1,250
- 4. % Crecimiento: +5.2%
- 5. % Cumplimiento: 98%

Entregables del Checkpoint 4

- ☐ Informe publicado en Power BI Service
- ☐ Workspace "TechStore BI Corporativo" creado
- ☐ Roles RLS configurados y probados
- ☐ App "TechStore Analytics" publicada
- ☐ Actualización automática programada
- ☐ Ficha técnica del sistema documentada
- ☐ Guía rápida para usuarios

Checklist de calidad

Antes de finalizar, verifica:

Aspecto	Verificación
Datos correctos	SUM de ingresos coincide con SQLite
Medidas precisas	KPIs sin errores (DIVIDE maneja cero)
Filtros funcionan	Slicers afectan a todos los gráficos
RLS aplica	Cada rol ve solo sus datos
Actualización OK	Gateway conectado y programado
App publicada	Usuarios pueden acceder
Documentación	Ficha técnica completa

Próximos pasos tras el libro

1. **Conecta fuentes reales:** reemplaza SQLite por datos reales de tu empresa
2. **Expande el modelo:** agrega más tablas de hechos (inventario, marketing)
3. **Automatiza ETL:** pipeline Python completo (Libro 2) que alimente el star schema
4. **Crea más apps:** dashboards por departamento (RRHH, Finanzas, Operaciones)
5. **Implementa alertas:** usando Power Automate + Power BI
6. **Forma usuarios:** capacita al equipo para usar los dashboards

¡Felicidades! Has completado **Business Intelligence y Modelado de Datos**. Construíste un sistema de BI corporativo completo desde cero: desde el modelo dimensional hasta la publicación en la nube con seguridad y actualización automática.

La colección **Data & Analytics Mastery** continúa:

4. Ingeniería de Datos Moderna
5. Big Data, ML e IA: Especialización Avanzada

Conclusión

De los datos al dashboard corporativo

Has completado un viaje completo por el mundo del Business Intelligence. Empezaste entendiendo qué es BI y la arquitectura de datos, diseñaste un modelo dimensional estrella, construiste dashboards profesionales en Power BI Desktop, dominaste DAX y Power Query, implementaste seguridad con RLS, y finalmente publicaste todo en la nube como un sistema corporativo completo.

Lo que has logrado:

1. **Fundamentos de BI:** arquitectura de datos, modelado dimensional (estrella/copo de nieve), KPIs y métricas de negocio
2. **Power BI Desktop:** conexión de datos, modelado, relaciones, visualizaciones, formato condicional
3. **DAX y Power Query:** CALCULATE, FILTER, time intelligence, tablas calculadas, transformaciones ETL
4. **Modelo Semántico:** KPIs dinámicos con parámetros, análisis What-If, Pareto, tablas desconectadas
5. **Seguridad:** RLS por roles (Vendedor, Gerente, Ejecutivo), tabla de seguridad, USERPRINCIPALNAME
6. **Power BI Service:** publicación, workspaces, apps, programación de actualización, suscripciones

Habilidades adquiridas

Área	Skill
BI	Power BI Desktop, Service, modelado dimensional
Lenguajes	DAX (CALCULATE, FILTER, time intelligence), Power Query M
Modelado	Star schema, dimensiones, hechos, tablas auxiliares
Seguridad	RLS, roles, USERNAME, tabla de seguridad
Nube	Power BI Service, workspaces, apps, gateway, actualización automática
Gobernanza	Documentación, certificación, estándares, auditoría

¿Y ahora qué?

La colección **Data & Analytics Mastery** continúa. Este libro es el tercero de cinco:

Libro 4: Ingeniería de Datos Moderna — Pipelines, Data Lakes, Orquestación

Libro 5: Big Data, ML e IA — Especialización Avanzada

Mientras tanto, aquí tienes ideas para seguir practicando:

1. **Conecta una fuente real** (ERP, CRM, Google Analytics) a tu modelo Power BI
2. **Crea dashboards departamentales** (RRHH: rotación, Finanzas: P&L, Marketing: campañas)
3. **Implementa alertas automáticas** con Power Automate + Power BI
4. **Desarrolla un modelo de IA** con AutoML o Python para predecir ventas
5. **Expande TechStore:** agrega tablas de inventario, devoluciones, costes de envío

Reflexión final

TechStore comenzó como un archivo SQLite y hoy tiene un sistema de BI corporativo completo en la nube, con seguridad por roles, actualización automática y un portal de dashboards accesible para toda la organización.

La diferencia entre un analista que usa Excel y un profesional de BI no está en las herramientas, sino en la forma de pensar: modelar datos dimensionalmente, definir métricas con precisión, asegurar que cada usuario vea solo lo que debe ver, y diseñar sistemas que escalen con el negocio.

Eso es lo que has aprendido en este libro. No solo a mover botones en Power BI, sino a diseñar soluciones de inteligencia de negocio profesionales.

Gracias por acompañarme en este viaje. Nos vemos en el Libro 4: *Ingeniería de Datos Moderna*.

— Alex Goyzueta Delgado

Apéndice A: Soluciones a los Ejercicios

Capítulo 1: Introducción al BI

Ejercicio 10 — Definiciones clave:

1. **Business Intelligence:** disciplina que combina herramientas, procesos y tecnologías para transformar datos brutos en información útil para la toma de decisiones empresariales.
2. **OLTP:** sistemas transaccionales optimizados para escritura (ej: ERP). **OLAP:** sistemas analíticos optimizados para consultas complejas (ej: Power BI).
3. **Modelo dimensional:** técnica de modelado que organiza datos en tablas de hechos (métricas) y dimensiones (contexto), optimizada para consultas analíticas.

Capítulo 2: Arquitectura de Datos

Ejercicio 8 — Componentes de la arquitectura TechStore:

Componente	Herramienta	Propósito
Fuente transaccional	SQLite	Datos operacionales
ETL/ELT	Power Query	Limpieza y transformación
Data Warehouse	Star schema (SQLite)	Modelo dimensional analítico
BI	Power BI Desktop	Modelado y visualización
Distribución	Power BI Service	Portal corporativo cloud

Capítulo 3: Modelado Dimensional

Ejercicio 8 — Identificar grano de las tablas:

```
-- Grano: 1 fila = 1 pedido
SELECT order_id, COUNT(*) FROM orders GROUP BY order_id;

-- Grano: 1 fila = 1 línea de pedido
SELECT order_id, product_id, COUNT(*)
FROM order_items GROUP BY order_id, product_id;
```

Ejercicio 9 — Modelo estrella SQL:

```
CREATE TABLE dim_date (
    date_sk INTEGER PRIMARY KEY,
    full_date DATE NOT NULL,
```

```

        year INTEGER,
        month INTEGER,
        month_name TEXT,
        quarter INTEGER,
        day_of_week INTEGER
    );

CREATE TABLE dim_customer (
    customer_sk INTEGER PRIMARY KEY,
    customer_id INTEGER,
    name TEXT,
    email TEXT,
    city TEXT,
    country TEXT,
    registration_date DATE
);

CREATE TABLE dim_product (
    product_sk INTEGER PRIMARY KEY,
    product_id INTEGER,
    name TEXT,
    category TEXT,
    unit_price REAL
);

CREATE TABLE dim_employee (
    employee_sk INTEGER PRIMARY KEY,
    employee_id INTEGER,
    name TEXT,
    email TEXT,
    region TEXT,
    role TEXT
);

CREATE TABLE fact_orders (
    order_sk INTEGER PRIMARY KEY,
    order_id INTEGER,
    date_sk INTEGER,
    customer_sk INTEGER,
    employee_sk INTEGER,
    total REAL,
    status TEXT,
    FOREIGN KEY (date_sk) REFERENCES dim_date(date_sk),
    FOREIGN KEY (customer_sk) REFERENCES dim_customer(customer_sk),
    FOREIGN KEY (employee_sk) REFERENCES dim_employee(employee_sk)
);

CREATE TABLE fact_order_items (
    item_sk INTEGER PRIMARY KEY,
    order_sk INTEGER,
    product_sk INTEGER,
    quantity INTEGER,

```

```

unit_price REAL,
FOREIGN KEY (order_sk) REFERENCES fact_orders(order_sk),
FOREIGN KEY (product_sk) REFERENCES dim_product(product_sk)
);

```

Capítulo 4: KPIs y Métricas

Ejercicio 1 — KPI vs métrica:

- **KPI:** métrica crítica vinculada a un objetivo estratégico (ej: "% Crecimiento de ingresos" con objetivo > 10%)
- **Métrica:** medida cuantitativa sin objetivo estratégico directo (ej: "Total de pedidos del mes")
- **Diferencia:** un KPI tiene un objetivo y una decisión asociada

Ejercicio 5 — Ficha técnica KPI:

```

# Ficha Técnica: Ticket Promedio
Nombre: Ticket Promedio
Fórmula: AVERAGE(fact_orders[total])
Frecuencia: Mensual
Responsable: Analista BI
Objetivo: > $300
Fuente: fact_orders

```

Capítulo 5: Power BI — Primeros Pasos

Ejercicio 7 — Consulta SQL de conexión:

```

SELECT
    o.order_id,
    o.order_date,
    o.total,
    o.status,
    c.name AS customer_name,
    c.email,
    c.city,
    c.country
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.order_date >= '2024-01-01';

```

Capítulo 6: Modelado en Power BI

Ejercicio 6 — Cardinalidades:

- `dim\_product` `fact\_order\_items`: 1:* (un producto aparece en muchas líneas)
- `dim\_customer` `fact\_orders`: 1:* (un cliente hace muchos pedidos)
- Dirección del filtro: de dimensión a hecho (única)

Capítulo 7: Visualizaciones en Power BI

Ejercicio 8 — Gráfico de barras apiladas:

1. Eje X: `dim\_product[category]`
2. Eje Y: `SUM(fact\_orders[total])`
3. Leyenda: `dim\_date[year]`
4. Ordenar: `category` ascendente

Capítulo 8: DAX Básico

Ejercicio 5 — Medidas:

```
-- Medida: Margen Bruto
Margen Bruto = SUM(fact_orders[total]) - SUM(fact_order_items[total_cost])

-- Medida: % Margen
% Margen = DIVIDE([Margen Bruto], SUM(fact_orders[total]), 0)

-- Medida con condición
Pedidos Altos =
CALCULATE(
    COUNTROWS(fact_orders),
    fact_orders[total] > 500
)
```

Capítulo 9: DAX Avanzado

Ejercicio 7 — CALCULATE + time intelligence:

```
-- Ventas mismo trimestre año anterior
Ventas Mismo Trimestre Año Anterior =
CALCULATE(
    [Ingresos Totales],
    SAMEPERIODLASTYEAR(dim_date[full_date])
)

-- Crecimiento trimestral YoY
Crecimiento YoY Trimestral =
VAR ventas_actual = [Ingresos Totales]
VAR ventas_anterior = [Ventas Mismo Trimestre Año Anterior]
RETURN DIVIDE(ventas_actual - ventas_anterior, ventas_anterior, 0)
```

Capítulo 10: Power Query ETL

Ejercicio 6 — Merge en Power Query M:

```
let
    Origen = Sql.Database("localhost", "techstore_bi"),
    Orders = Origen{[Schema="main",Item="orders"]}[Data],
    Customers = Origen{[Schema="main",Item="customers"]}[Data],
    Merge = Table.NestedJoin(
```

```

        Orders, {"customer_id"},
        Customers, {"customer_id"},
        "Customers", JoinKind.LeftOuter
    ),
    Expand = Table.ExpandRecordColumn(
        Merge, "Customers",
        {"name", "email", "city"},
        {"customer_name", "email", "city"}
    )
in
    Expand

```

Capítulo 11: Data Warehousing

Ejercicio 5 — SCD Tipo 2:

```

-- Insertar nueva versión (SCD Tipo 2)
UPDATE dim_customer
SET valid_to = CURRENT_DATE,
    is_current = 0
WHERE customer_id = 123 AND is_current = 1;

INSERT INTO dim_customer (
    customer_sk, customer_id, name, email,
    city, valid_from, valid_to, is_current
) VALUES (
    (SELECT COALESCE(MAX(customer_sk), 0) + 1 FROM dim_customer),
    123, 'Alex Goyzueta', 'alex@nuevoemail.com',
    'Lima', CURRENT_DATE, '9999-12-31', 1
);

```

Capítulo 12: Medidas Avanzadas

Ejercicio 8 — Matriz semáforo formato condicional:

```

-- Regla de formato condicional (color de fondo)
Color Semáforo =
SWITCH(
    TRUE(),
    [% Cumplimiento Ingresos] >= 1, "Green",
    [% Cumplimiento Ingresos] >= 0.8, "Yellow",
    "Red"
)

```

Capítulo 13: Power BI Service

Ejercicio 4 — Informe vs Panel:

- **Informe:** vista interactiva con múltiples páginas, filtros compartidos, drill-down
- **Panel:** vista resumen con KPIs fijos (pin de un informe), sin filtros interactivos

Capítulo 14: Seguridad RLS

Ejercicio 5 — RLS vs OLS:

- **RLS (Row-Level Security)**: filtra filas completas según el usuario
- **OLS (Object-Level Security)**: oculta columnas o tablas enteras
- OLS requiere **Premium** o **Premium Per User**

Capítulo 15: Proyecto Final

Ejercicio 8 — Plan de mantenimiento mensual:

1. **Semana 1**: verificar actualizaciones automáticas, revisar logs
2. **Semana 2**: validar calidad de datos (nulos, duplicados, outliers)
3. **Semana 3**: recoger feedback de usuarios, priorizar mejoras
4. **Semana 4**: implementar cambios, documentar, desplegar

Apéndice B: Cheatsheets

DAX — Medidas

```
// ===== Agregaciones =====
SUM(tabla[columna])
AVERAGE(tabla[columna])
COUNTROWS(tabla)
DISTINCTCOUNT(tabla[columna])
MIN(tabla[columna])
MAX(tabla[columna])

// ===== Filtros =====
CALCULATE(medida, filtro1, filtro2, ...)
FILTER(tabla, condición)
ALL(tabla) // Quita todos los filtros
ALLEXCEPT(tabla, col) // Quita todos excepto col
ALLSELECTED(tabla) // Quita filtros excepto externos
KEEPFILTERS(tabla) // Intersecta filtros
REMOVEFILTERS(tabla) // Elimina filtros

// ===== Time Intelligence =====
TOTALYTD(medida, fecha)
TOTALQTD(medida, fecha)
TOTALMTD(medida, fecha)
SAMEPERIODLASTYEAR(fecha)
DATEADD(fecha, -1, YEAR)
DATESINPERIOD(fecha, fin, -30, DAY)
DATESMTD(fecha)
DATESQTD(fecha)
DATESYTD(fecha)
PREVIOUSMONTH(fecha)
PREVIOUSQUARTER(fecha)
PREVIOUSYEAR(fecha)
PARALLELPERIOD(fecha, -1, MONTH)

// ===== Lógica =====
IF(condición, valor_sí, valor_no)
SWITCH(expr, val1, res1, val2, res2, ... default)
SWITCH(TRUE(), cond1, res1, cond2, res2, ... default)
AND(a, b)
OR(a, b)
```

```

NOT(condición)

// ===== Iteradores =====
SUMX(tabla, expresión)
AVERAGEX(tabla, expresión)
COUNTX(tabla, expresión)
MAXX(tabla, expresión)
MINX(tabla, expresión)
RANKX(tabla, medida)
FILTER(tabla, condición) // Filtra filas
ADDCOLUMNS(tabla, "col", expr) // Agrega columna

// ===== Variables =====
VAR nombre = expresión
RETURN resultado

// ===== Relaciones =====
RELATED(dim[col]) // Desde tabla actual a dimensión
RELATEDTABLE(hecho) // Desde dimensión a hecho
USERELATIONSHIP(dim[col], hecho[col]) // Relación inactiva

// ===== Valores =====
SELECTEDVALUE(dim[col], default)
VALUES(dim[col])
DISTINCT(dim[col])
HASONEVALUE(dim[col])
ISFILTERED(dim[col])
ISCROSSFILTERED(dim[col])

// ===== Seguridad =====
USERNAME()
USERPRINCIPALNAME()

```

Power Query M

```

// ===== Conexión =====
let
    Origen = Sql.Database("server", "db_name"),
    Tabla = Origen[Schema="schema",Item="table"][Data],
    CSV = Csv.Document(File.Contents("archivo.csv"), [Delimiter=","]),
    Excel = Excel.Workbook(File.Contents("archivo.xlsx"), null, true),
    Folder = Folder.Files("C:\datos\")
in
    Tabla

// ===== Transformación =====
Table.SelectRows(tabla, each [columna] > 100)
Table.RemoveColumns(tabla, {"col1", "col2"})
Table.RenameColumns(tabla, {"old", "new"})
Table.TransformColumnTypes(tabla, {"col", type number})
Table.Sort(tabla, {"col", Order.Descending})

```

```

Table.Distinct(tabla)
Table.AddColumn(tabla, "nueva", each [a] * [b])
Table.ReplaceValue(tabla, "old", "new", Replacer.ReplaceText, {"col"})
Table.FillDown(tabla, {"col"})

// ===== Merge =====
Table.NestedJoin(t1, {"key"}, t2, {"key"}, "new_col", JoinKind.LeftOuter)
Table.ExpandTableColumn(tabla, "col", {"sub1", "sub2"}, {"sub1", "sub2"})
Table.Combine({t1, t2}) // Append

// ===== Fechas =====
Date.Year([fecha])
Date.Month([fecha])
Date.MonthName([fecha])
Date.Quarter([fecha])
Date.DayOfWeek([fecha])
Date.DayOfWeekName([fecha])
Date.StartOfMonth([fecha])
Date.EndOfMonth([fecha])
Date.AddDays([fecha], 30)
Duration.Days([fecha_fin] - [fecha_ini])

// ===== Texto =====
Text.Trim([col])
Text.Lower([col])
Text.Upper([col])
Text.Proper([col])
Text.Replace([col], "a", "b")
Text.Start([col], 5)
Text.End([col], 3)

// ===== Números =====
Number.Round([col], 2)
Number.Abs([col])
Number.IntegerDivide([a], [b])
Number.Mod([a], [b])

```

SQL para BI

```

-- Dimensiones
SELECT DISTINCT customer_id, name, email, city
FROM customers;

-- Hechos con dimensiones
SELECT
    o.order_id,
    o.order_date,
    c.name AS customer,
    c.city,
    p.name AS product,
    p.category,

```

```

        oi.quantity,
        oi.unit_price,
        oi.quantity * oi.unit_price AS line_total,
        o.total
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id;

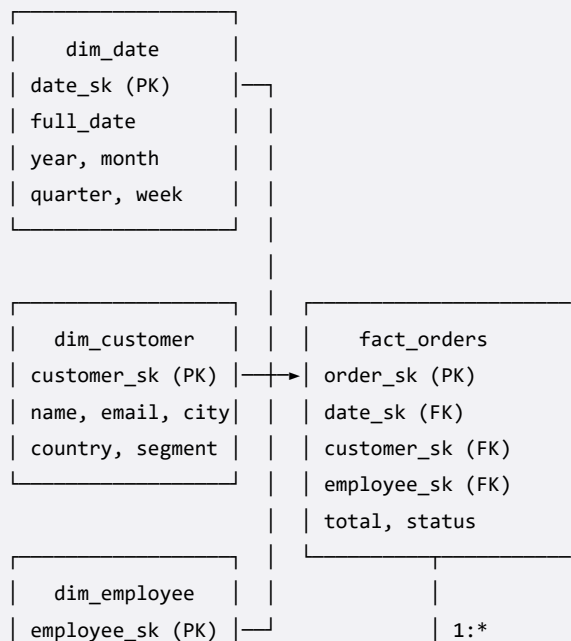
-- Agregaciones
SELECT
    c.city,
    COUNT(DISTINCT o.order_id) AS pedidos,
    SUM(o.total) AS ingresos,
    AVG(o.total) AS ticket_promedio
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.city
ORDER BY ingresos DESC;

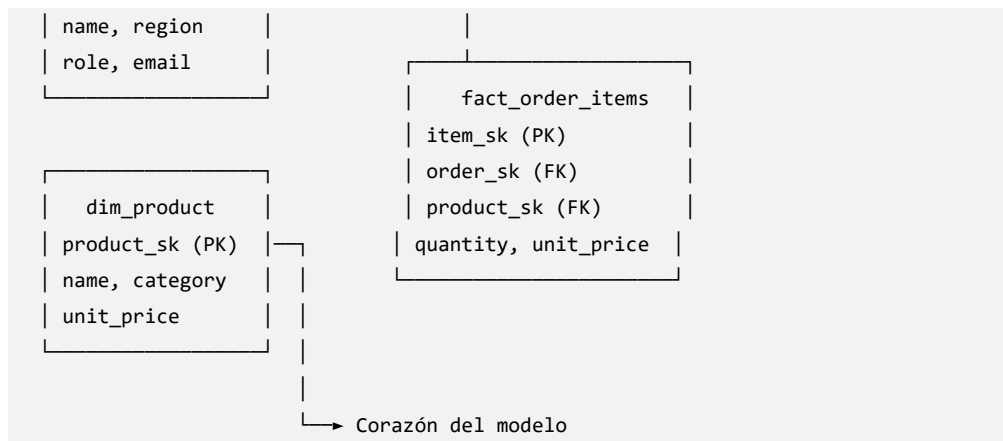
-- Ventas por período
SELECT
    strftime('%Y', o.order_date) AS anno,
    strftime('%m', o.order_date) AS mes,
    COUNT(*) AS pedidos,
    SUM(o.total) AS ingresos
FROM orders o
GROUP BY anno, mes
ORDER BY anno, mes;

```

Modelado Dimensional

// Star Schema





Power BI Service

```

// Publicar
Desktop → Publicar → Service

// Workspaces crean app
Workspace → Crear App → Publicar → Consumidores

// Roles en workspace
Admin:    todo
Member:  editar + publicar
Viewer:  solo ver

// RLS
Desktop:  Modelado → Administrar roles → DAX
Service:  Dataset → Seguridad nivel fila → Asignar

// Actualización
Dataset → Programar actualización → Frecuencia diaria

// Suscripciones
Informe → Suscribirse → Email + PDF adjunto
  
```

Conceptos clave de BI

Concepto	Definición
Star schema	Tabla de hechos + dimensiones alrededor
Snowflake schema	Dimensiones normalizadas en varias tablas
Grano	Nivel de detalle de una tabla de hechos
Cardinalidad	Tipo de relación (1:*, 1:1, *:*)
SCD	Dimensiones de cambio lento (Tipo 1, 2, 3)
ETL	Extract → Transform → Load
ELT	Extract → Load → Transform
MOLAP	Cubo multidimensional precalculado
ROLAP	Consultas directas a base relacional
DirectQuery	Power BI consulta origen en tiempo real

Import	Power BI carga datos en memoria (columna)
Composite	Mezcla DirectQuery + Import
Data Gateway	Puente entre Power BI Service y datos on-prem
Workspace	Contenedor de informes/datasets en Service
App	Pack de contenido publicado para consumidores

Apéndice C: Costes, Licencias y Recursos de Power BI

Planes y Costes de Power BI

Plan	Coste	Ideal para
Gratuito	\$0	Aprendizaje, crear informes para uno mismo
Pro	~\$10 USD/usuario/mes	Compartir informes, colaborar, workspaces
Premium Per User	~\$20 USD/usuario/mes	IA, paginados, mayor capacidad
Premium Capacity	Desde \$4,995 USD/mes	Empresas con >500 usuarios

¿Qué necesitas para este libro?

- Las **licencias gratuitas** bastan para aprender Power BI Desktop
- Para **publicar y compartir** en el servicio, necesitas al menos **Pro**
- Microsoft ofrece **cuenta de prueba gratuita por 60 días** de Power BI Pro

Free vs Pro vs Premium

Característica	Gratis	Pro	PPU
Power BI Desktop			
Publicar en Service	Solo Mi área		
Compartir informes			
Workspaces			
Apps			
RLS	(crear)	(asignar)	
Actualización programada		8/día	48/día
XMLA endpoints			
AI visuals			
Paginados			
Capacidad	1 GB	1 GB	100 TB

Configuración de Power BI Service

Paso 1: Crear cuenta Microsoft

1. Ve a `https://powerbi.microsoft.com``
2. "Comenzar gratis" usa tu email personal o corporativo

3. Completa el registro

Paso 2: Iniciar prueba Pro

1. En Power BI Service: icono de perfil "Iniciar prueba Pro"
2. 60 días gratis, sin necesidad de tarjeta

Paso 3: Instalar Power BI Desktop

1. Descarga de ``https://powerbi.microsoft.com/desktop``
2. Instalador (~400 MB) — Windows 10+
3. Power BI Desktop se actualiza mensualmente

Power BI Gateway

Necesario cuando: tus datos están en local (SQLite, Excel, SQL Server local) y quieres actualizarlos automáticamente en Power BI Service.

Instalación

1. Descarga de ``https://powerbi.microsoft.com/gateway``
2. Elige **modo personal** (para un solo usuario)
3. Registra con tu cuenta de Power BI
4. Conecta tus fuentes de datos locales

Alternativas para evitar Gateway

1. **Migrar a BigQuery** (como en Libro 2) — Power BI se conecta directo a BigQuery
2. **Usar OneDrive/SharePoint** — Power BI Service puede leer archivos Excel/CSV desde ahí
3. **Power BI Dataflows** — ETL en la nube, sin gateway

Troubleshooting común

Error: "No se pudo cargar el modelo"

- Solución: reduce el tamaño del modelo, elimina tablas innecesarias
- Límite Pro: 1 GB por dataset

Error: "RLS no funciona en el servicio"

- Solución: verifica que asignaste usuarios a los roles en "Seguridad a nivel de fila"
- Prueba con "Ver como" en el servicio

Error: "Actualización falló"

- Solución: verifica credenciales del gateway, conectividad SQLite, ruta de archivo
- Revisa el historial de actualización para ver el error específico

Error: "No tienes permiso para ver este informe"

- Solución: el informe está en un workspace. Pide acceso al admin
- O comparte el informe directamente desde el workspace

Error: "USERNAME() devuelve null"

- Solución: USERNAME() funciona en el servicio, no siempre en Desktop
- Para pruebas en Desktop, usa un valor fijo temporal

Recursos adicionales

- **Power BI Desktop (descarga):** <https://powerbi.microsoft.com/desktop>
- **Power BI Service:** <https://app.powerbi.com>
- **Documentación oficial:** <https://learn.microsoft.com/power-bi>
- **Galería de informes:** <https://community.powerbi.com> (ver ejemplos)
- **DAX Guide:** <https://dax.guide> (referencia completa de funciones)
- **DAX Patterns:** <https://www.daxpatterns.com> (patrones de medidas)
- **Power Query M Reference:** <https://learn.microsoft.com/powerquery-m>
- **Comunidad Power BI:** <https://community.fabric.microsoft.com>
- **YouTube oficial:** Microsoft Power BI (canal oficial)
- **SQLBI:** <https://www.sqlbi.com> (cursos avanzados de DAX — Alberto Ferrari y Marco Russo)

Plantilla de proyecto BI

Puedes descargar la plantilla del proyecto TechStore y adaptarla a tus datos:

1. **Archivo:** TechStore_BI.pbix (lo creaste durante el libro)
2. **Modelo:** star schema con 6 tablas
3. **Medidas:** 25+ medidas DAX documentadas
4. **RLS:** 3 roles configurados

Para usar la plantilla con tus datos:

1. Reemplaza las fuentes de datos (Power Query)
2. Ajusta las relaciones si cambian los nombres
3. Actualiza las medidas según tus KPIs
4. Configura RLS con tus usuarios y regiones

Sobre el Autor

Alex Goyzueta Delgado es analista de datos, instructor y emprendedor tecnológico con más de una década de experiencia transformando datos en decisiones de negocio.

Su trayectoria abarca desde bases de datos relacionales y hojas de cálculo hasta plataformas cloud empresariales y Business Intelligence corporativo, trabajando con startups y corporaciones en Latinoamérica para implementar soluciones de analytics que generan impacto real.

Fundador de **Data & Analytics Academy**, donde ha formado a más de 5000 estudiantes en análisis de datos, SQL, Python, cloud computing y BI a través de cursos prácticos y project-based learning.

Cree firmemente en que la mejor forma de aprender datos es construyendo proyectos reales desde el día uno. Su metodología combina fundamentos técnicos sólidos con casos de negocio prácticos, preparando a sus estudiantes no solo para usar herramientas, sino para pensar como analistas y profesionales de datos.

Este libro es el tercero de la colección **Data & Analytics Mastery**, una serie diseñada para llevar al lector desde cero hasta analytics professional.

Conéctate con el autor:

- LinkedIn: <https://www.linkedin.com/in/alexgoyzueta>
- GitHub: <https://github.com/alexgoyzueta>

Otros libros de la colección:

1. *Los Fundamentos del Analista de Datos* — Domina SQL, Excel y Python desde cero
2. *Python para Datos y tu Primer Data Warehouse Cloud* — De Pandas a BigQuery
3. *Business Intelligence y Modelado de Datos* — Este libro
4. *Ingeniería de Datos Moderna* — Próximamente
5. *Big Data, ML e IA: Especialización Avanzada* — Próximamente